

EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Announcement

Midterm: March 10th

Review: March 3rd

Coverage: Every taught until GNN

Open book. More details are introduced in review lecture.

Agenda

- Recurrent Neural Networks (RNN)
 - What is RNN
 - Computation of RNN
 - Language Modeling
 - Long Short Term Memory (LSTM) Networks

Image captioning



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court

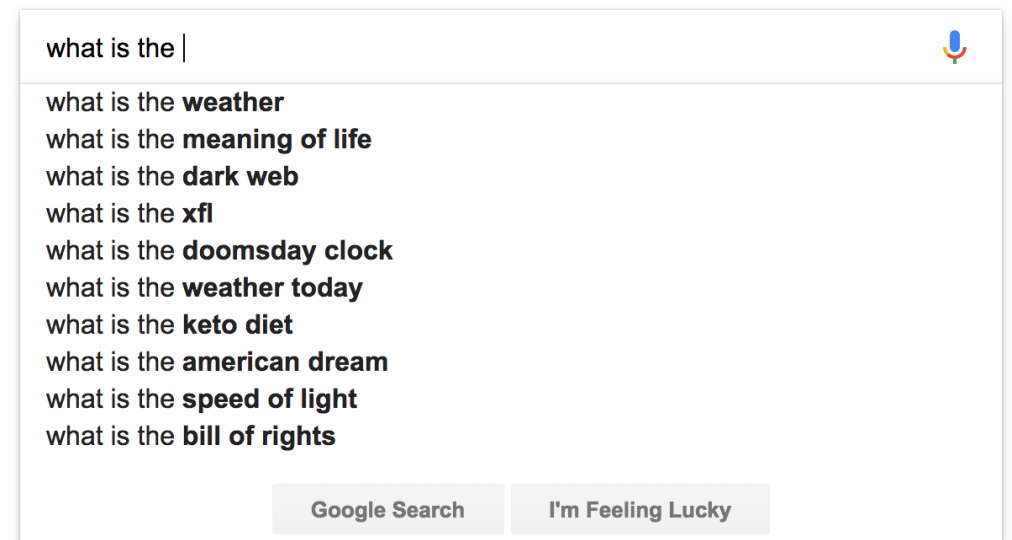
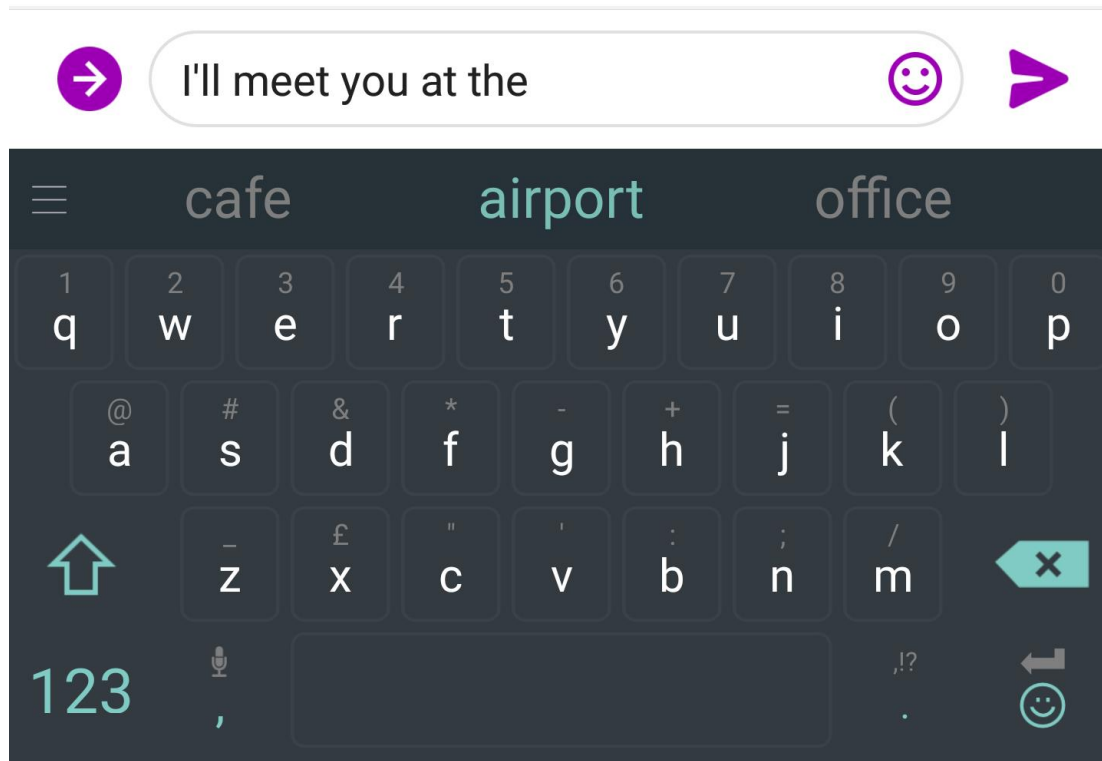


Two giraffes standing in a grassy field

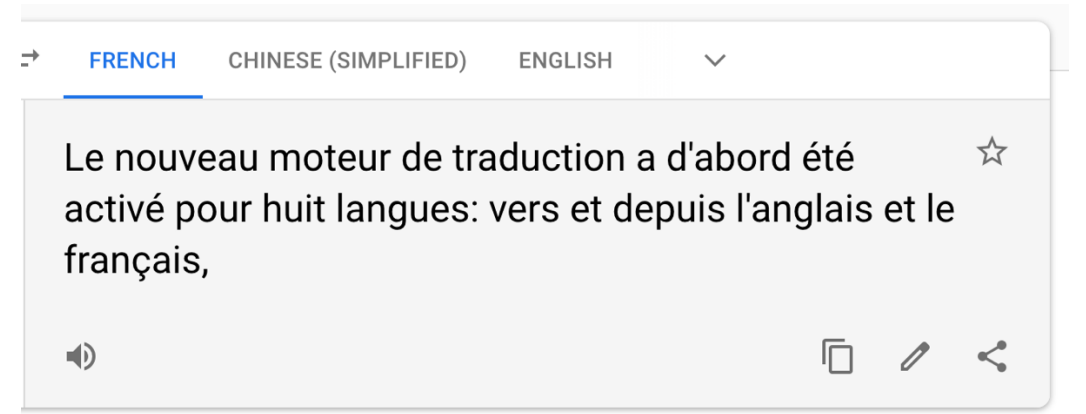
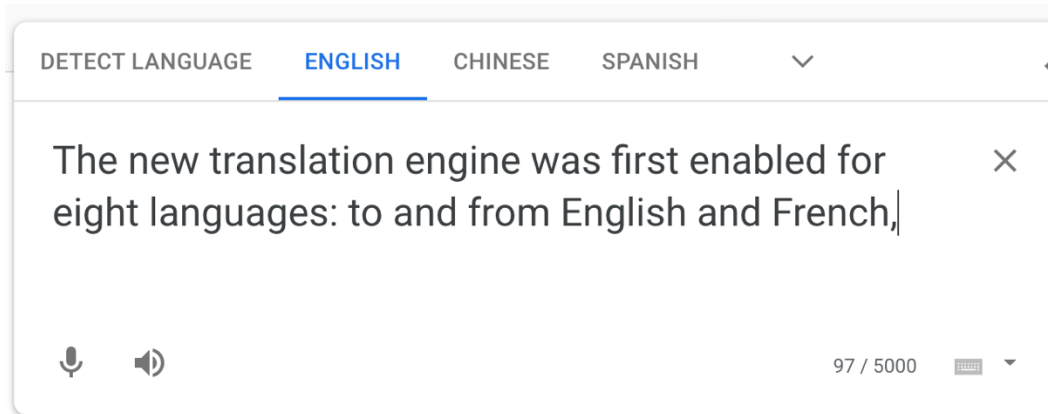


A man riding a dirt bike on a dirt track

Autocomplete prediction



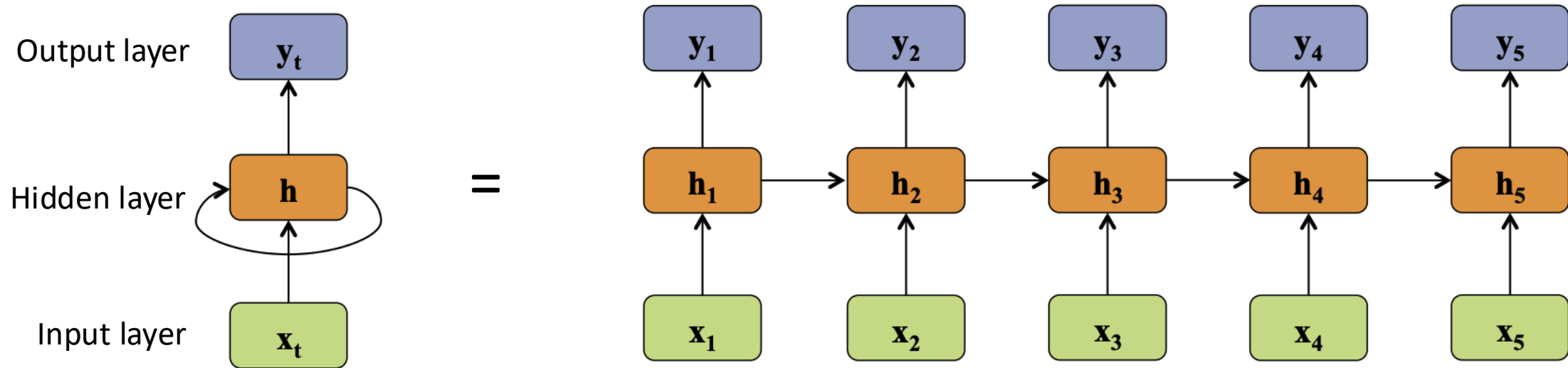
Machine translation



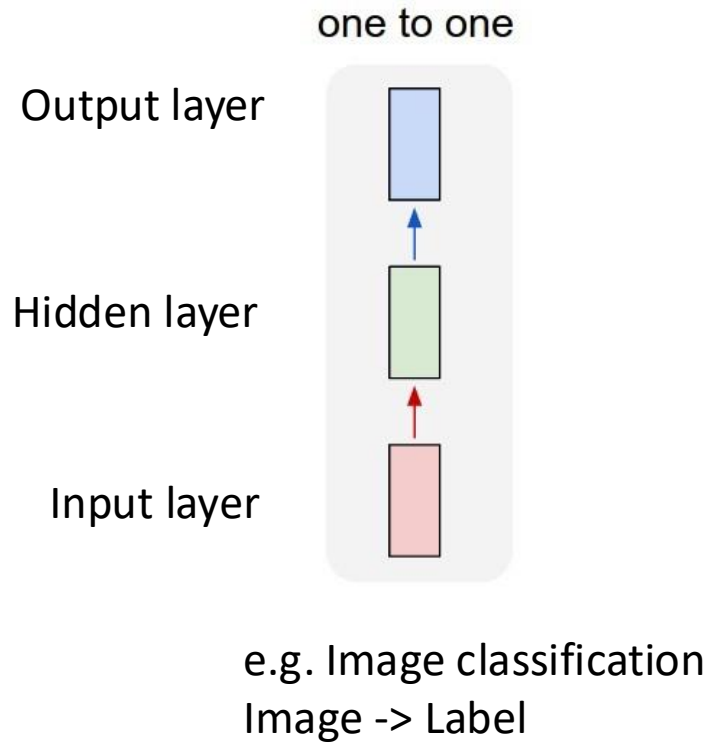
[Send feedback](#)

Recurrent Neural Networks (RNNs)

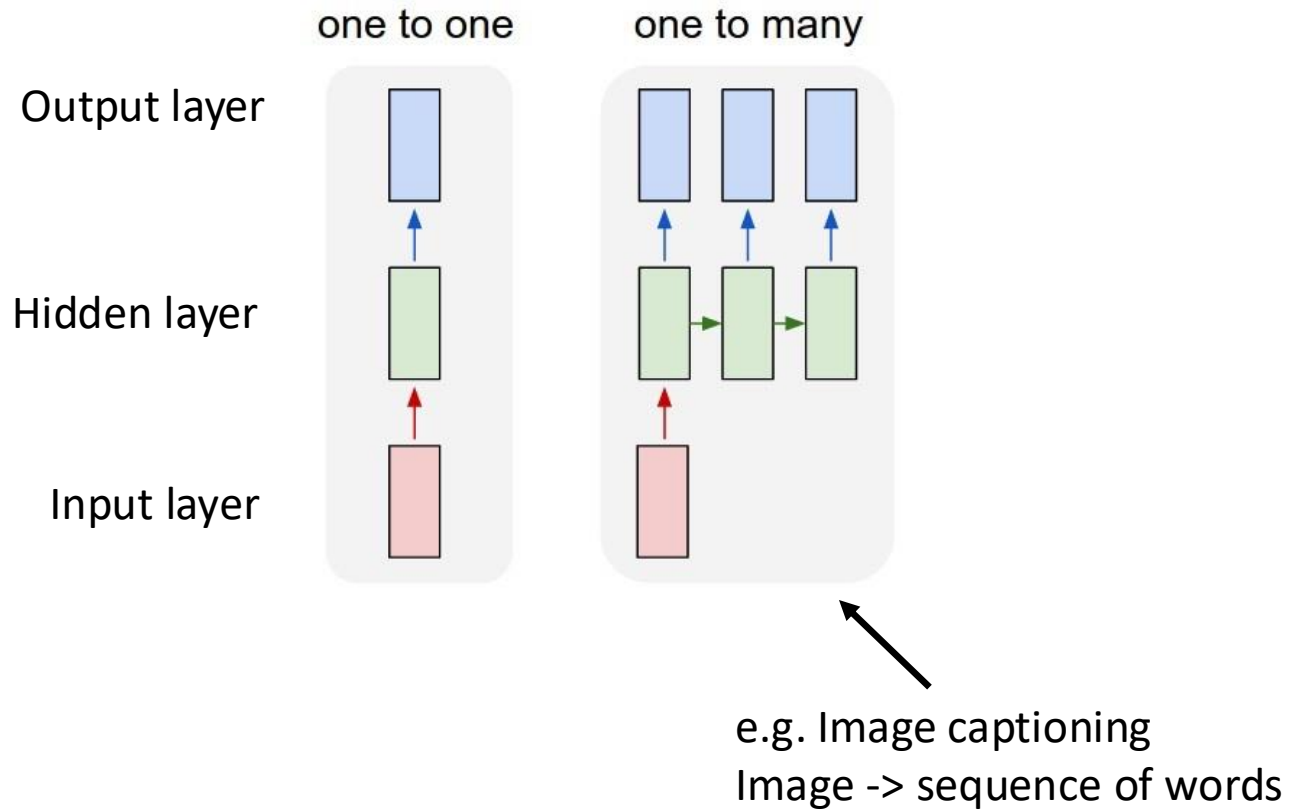
- RNNs are designed to handle **sequential information**
 - CNNs are designed to process image data
- RNNs introduce **state variables** to memorize past information, combine with the **current inputs**, to determine the current outputs



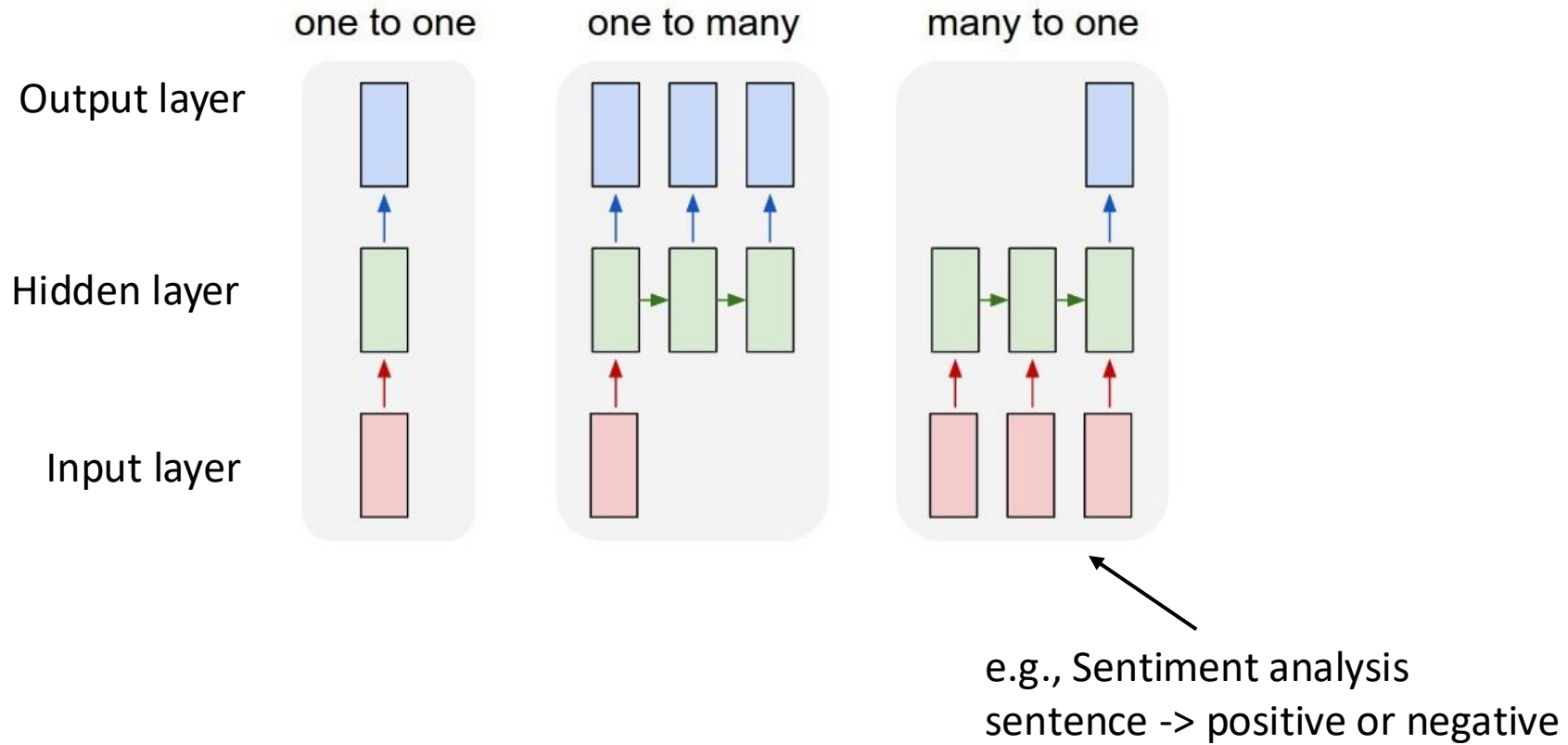
Ways to process sequences data



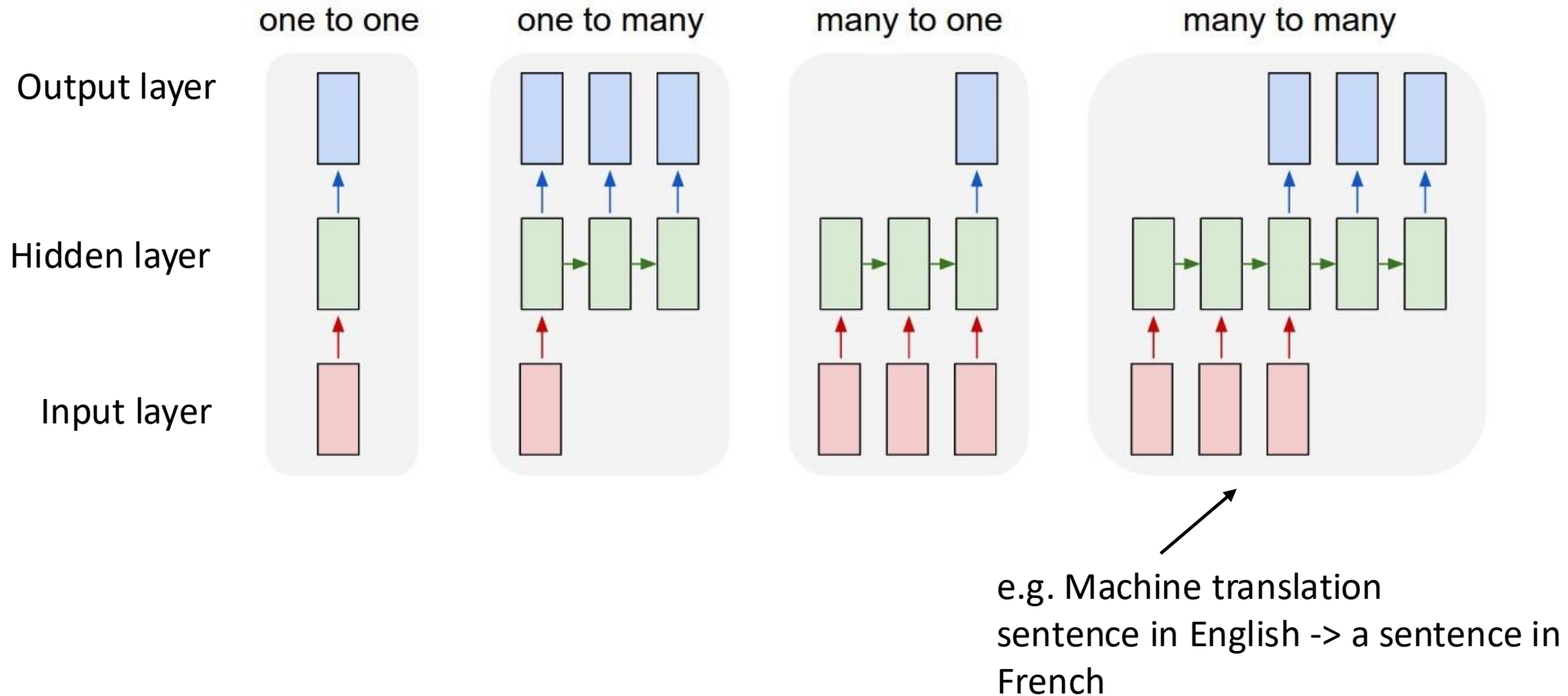
Ways to process sequences data



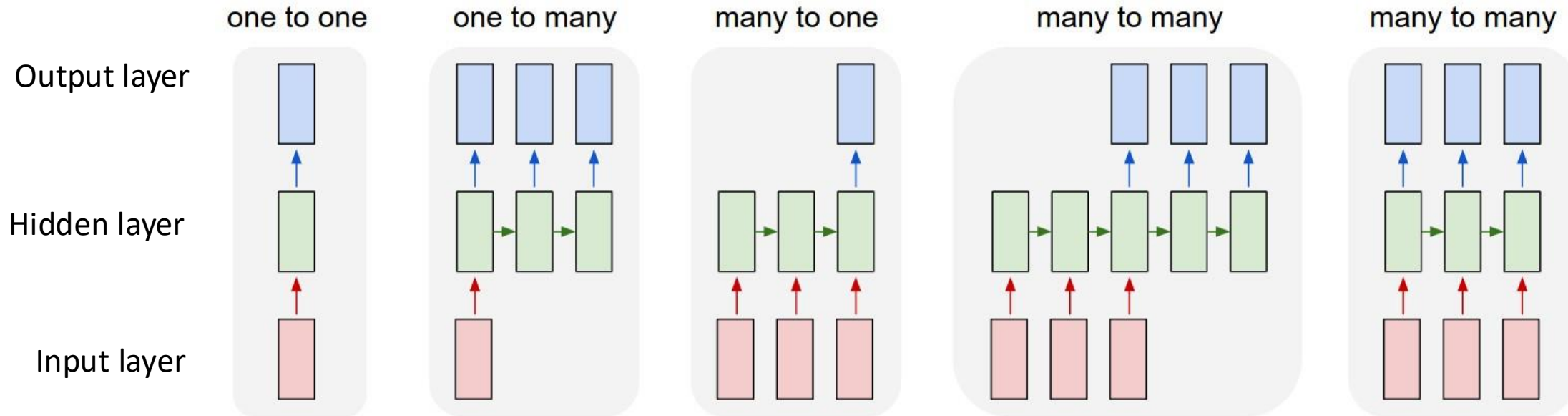
Ways to process sequences data



Ways to process sequences data



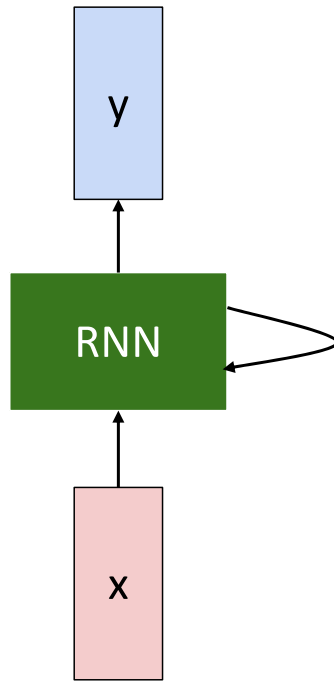
Ways to process sequences data



e.g., Per-frame video classification
Sequence of images -> Sequence of labels

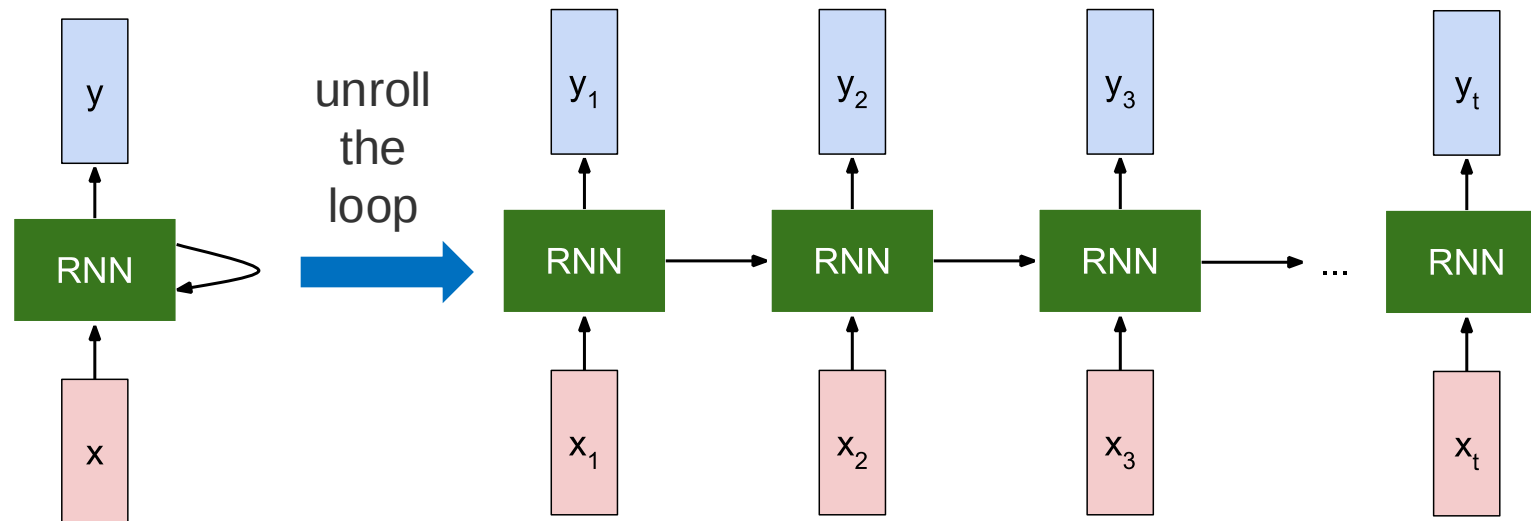
Same: they all maintain and update a “memory” during the sequence

How does RNN work



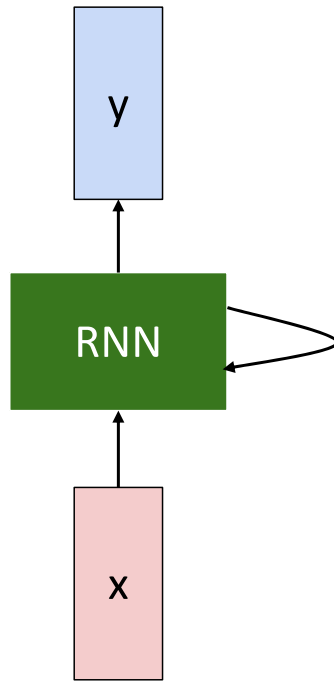
Key idea: RNNs have an “**internal state**” (hidden state) that is updated as a sequence is processed

Computation of RNN



A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor

Computation of RNN



We can process a sequence of vectors x by applying a recurrence formula at every time step:

new
state

activation
function

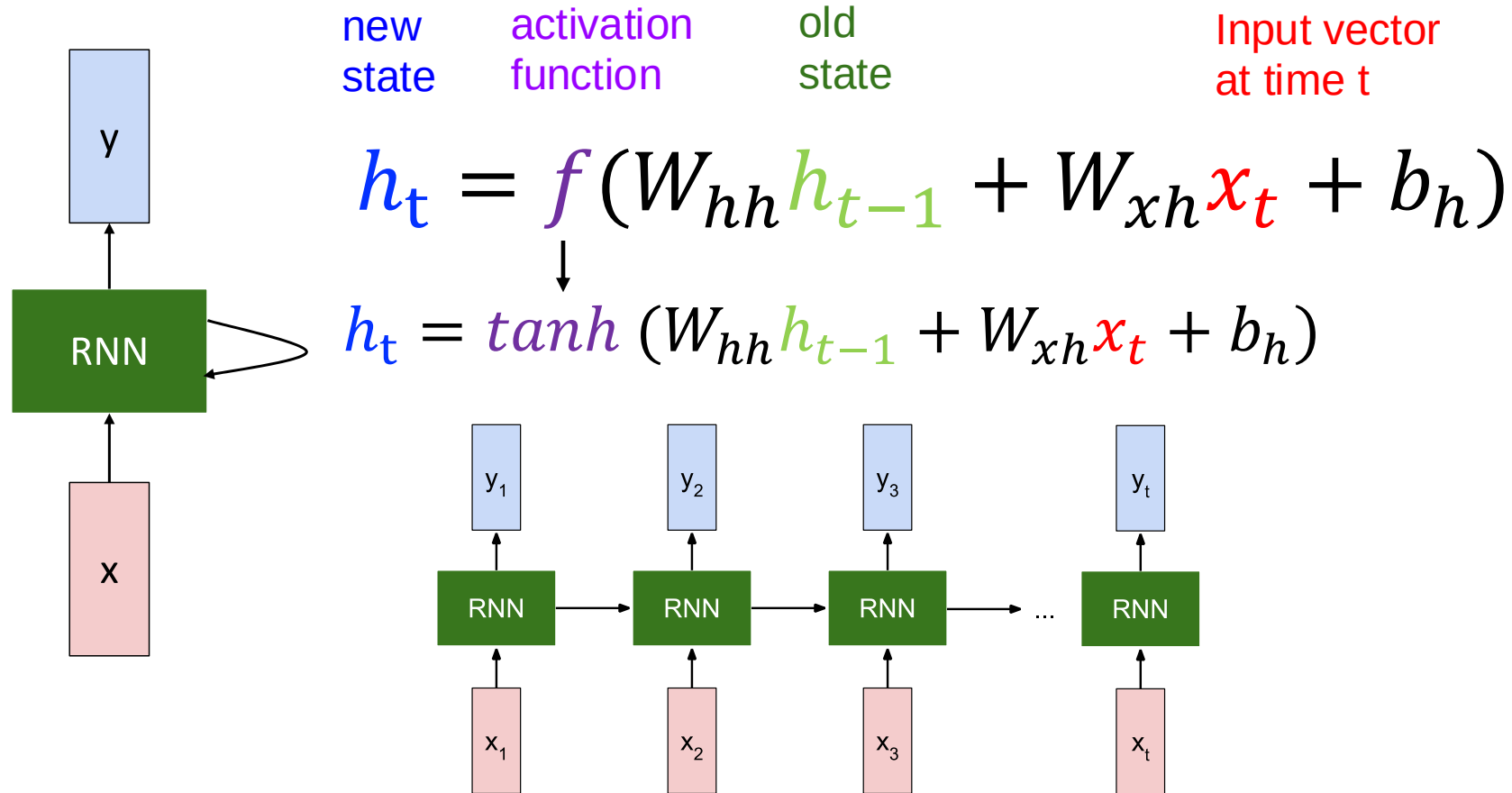
old
state

Input vector at
time t

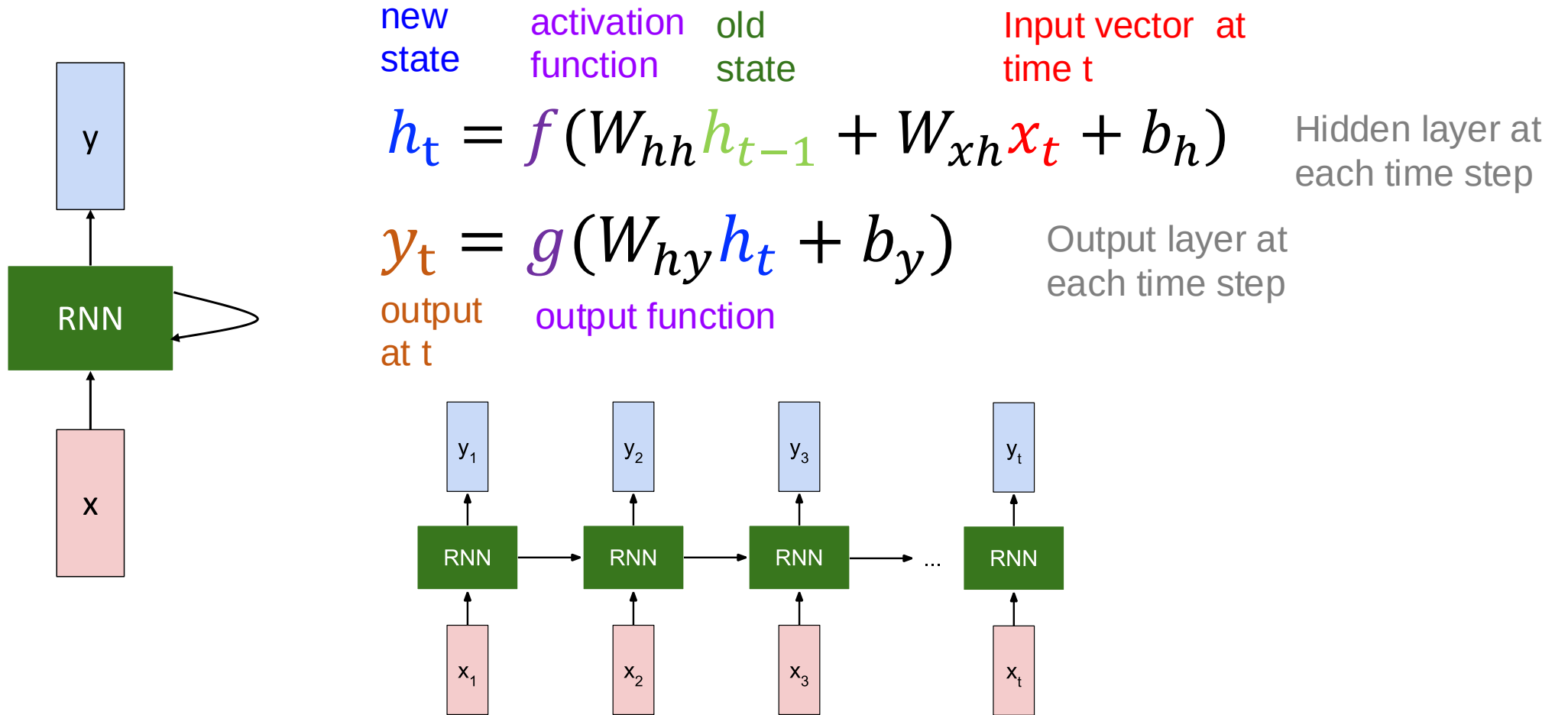
$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Notice: the same function and the same set of parameters are used at every time step.

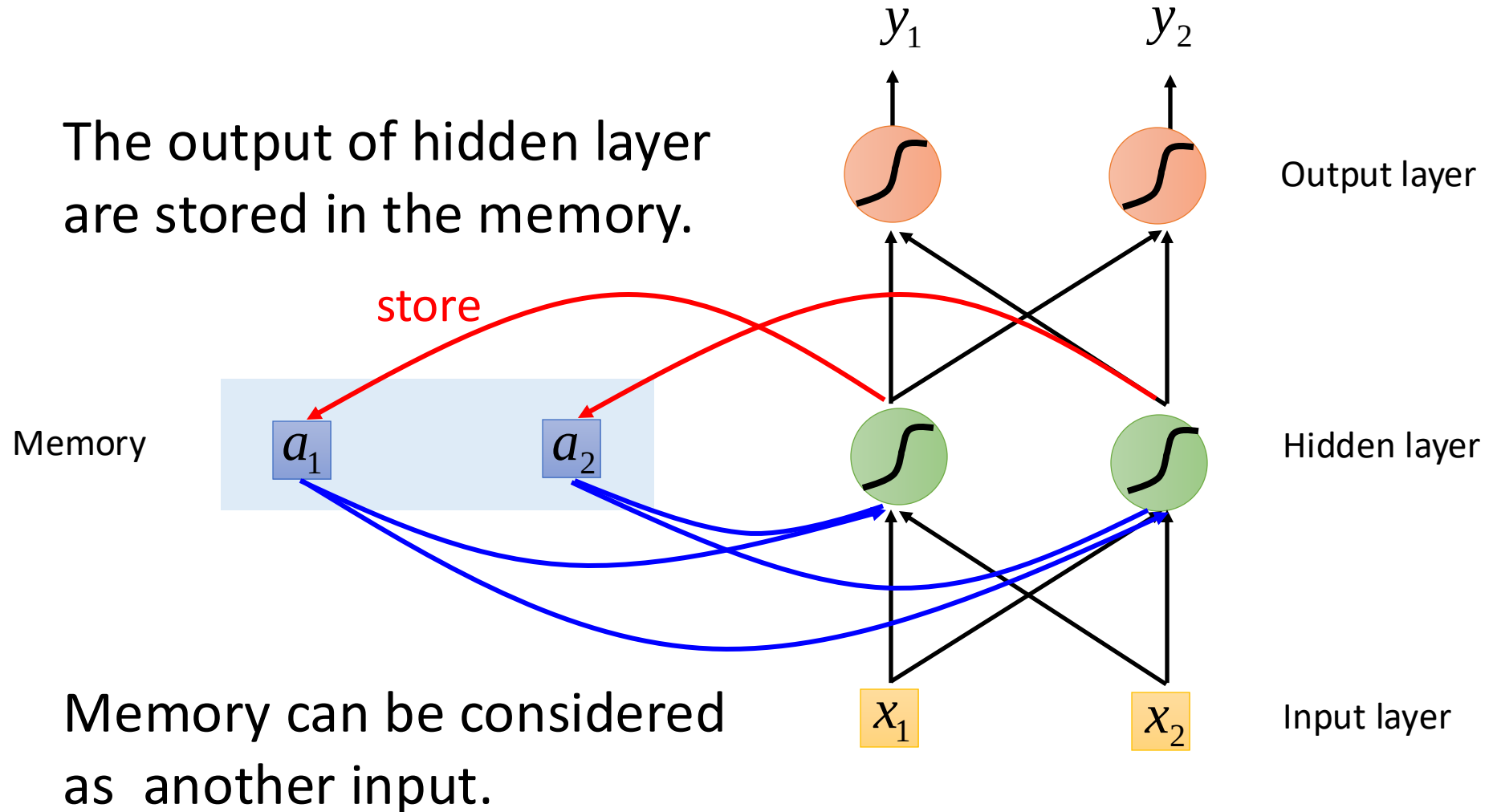
Computation of RNN



Recurrent Neural Networks



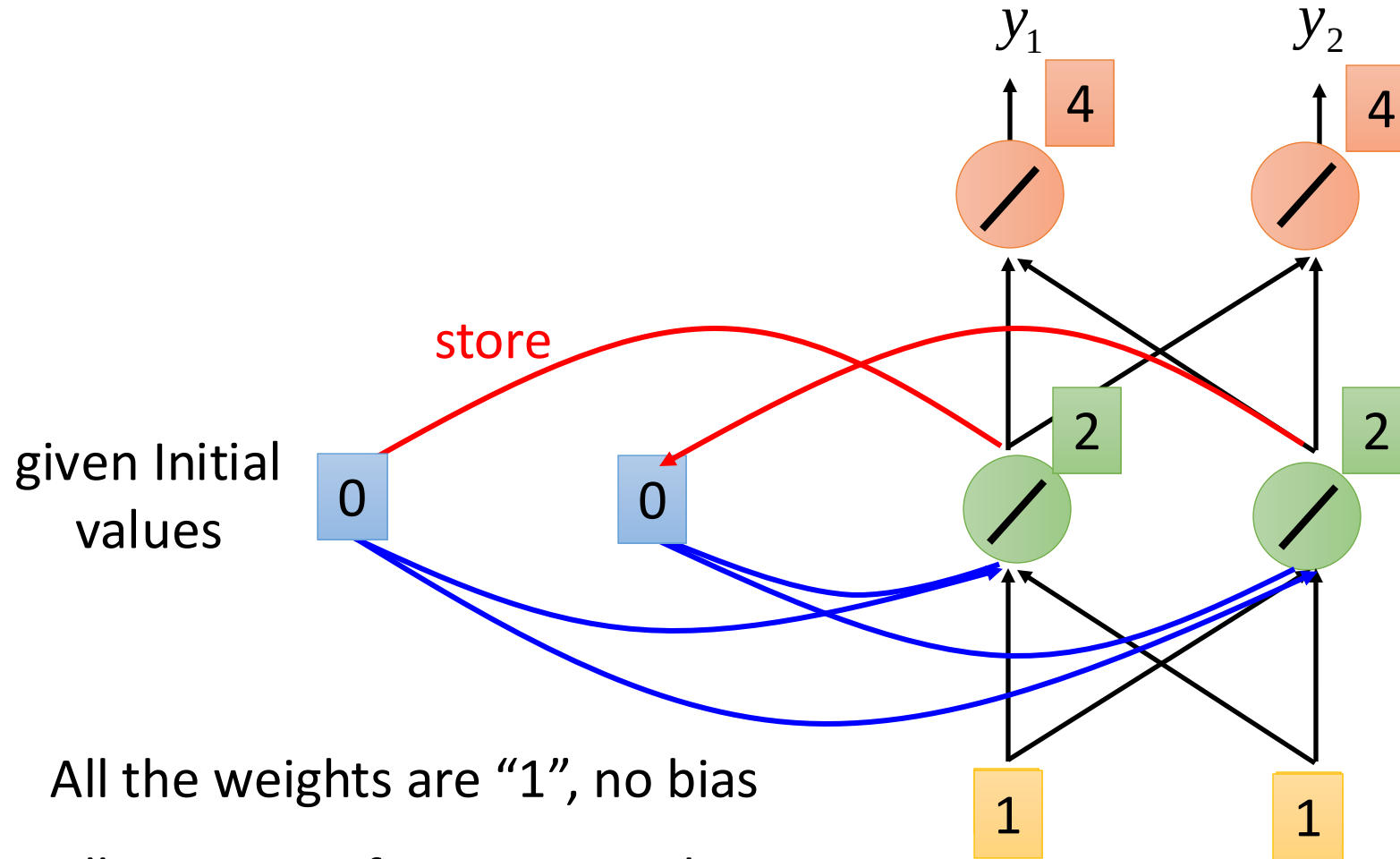
Example of RNN



Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$



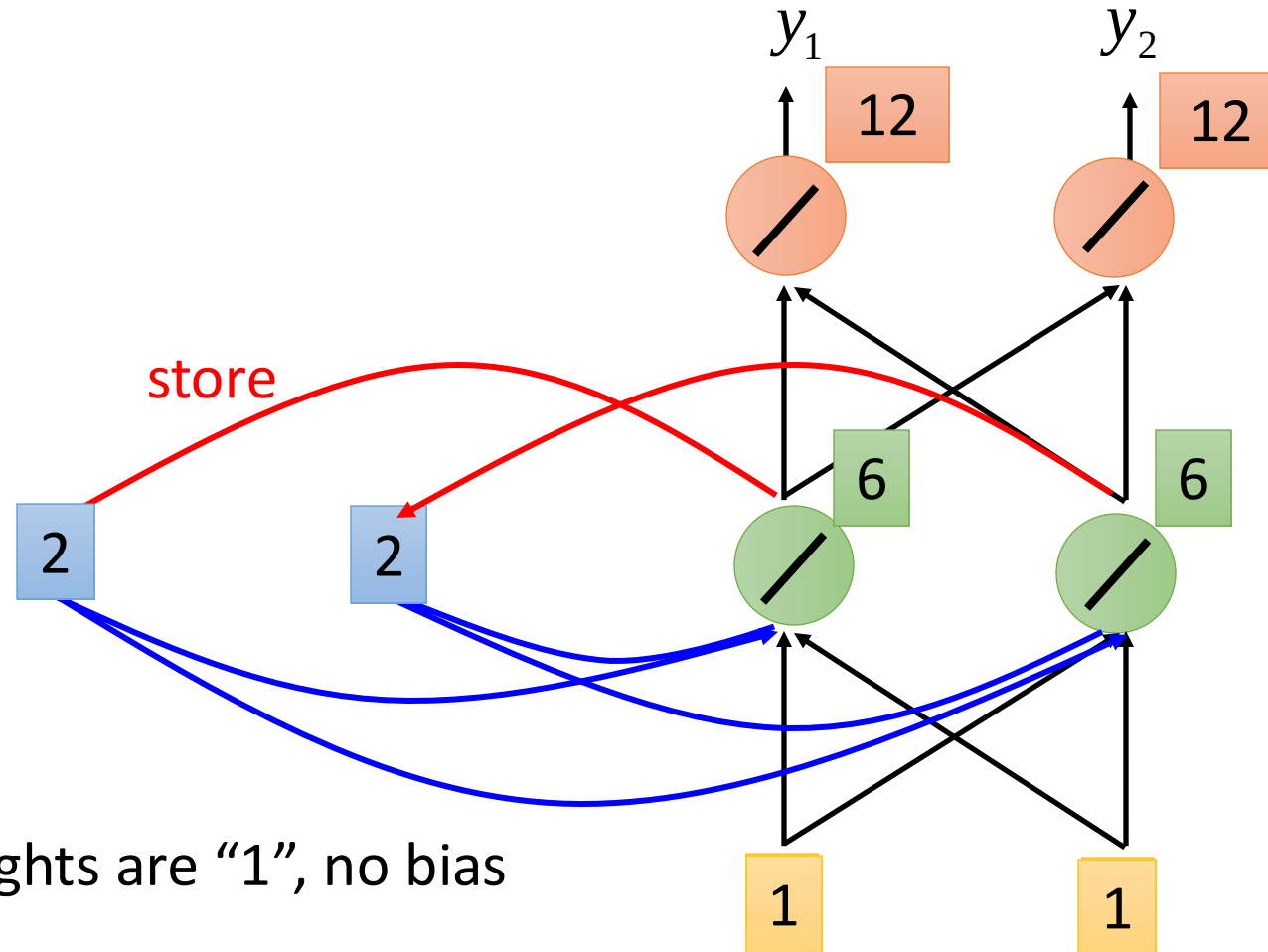
All the weights are "1", no bias

All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix}$



All the weights are "1", no bias

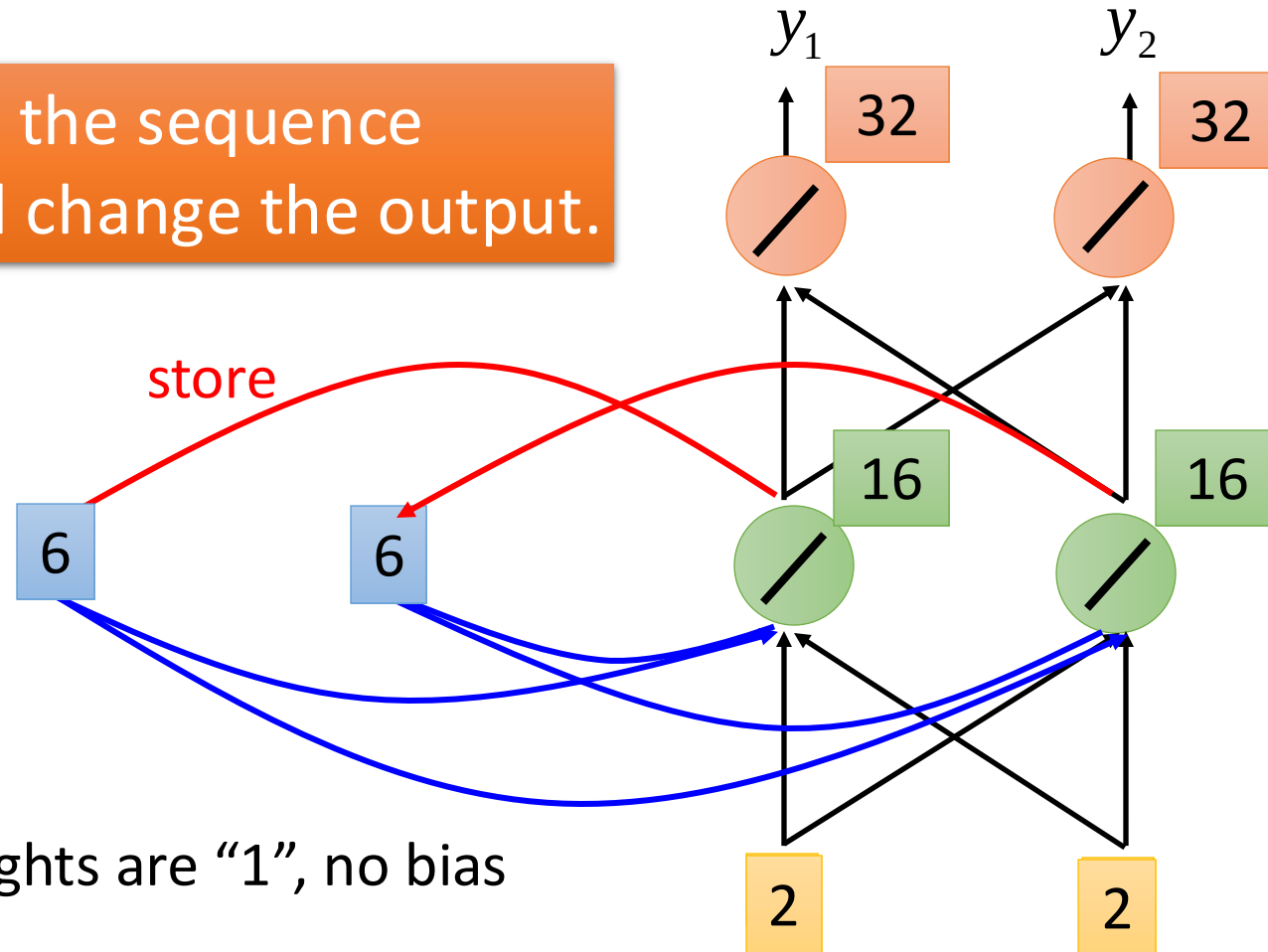
All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix} \begin{bmatrix} 32 \\ 32 \end{bmatrix}$

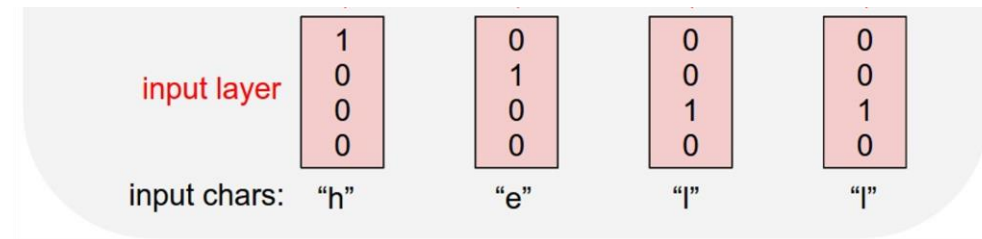
Changing the sequence order will change the output.



Example: language modeling

- A language model estimates a probability distribution of a linguistic unit (test, sentence, token, symbol)
 - It learn the rule of how people speak (or write)

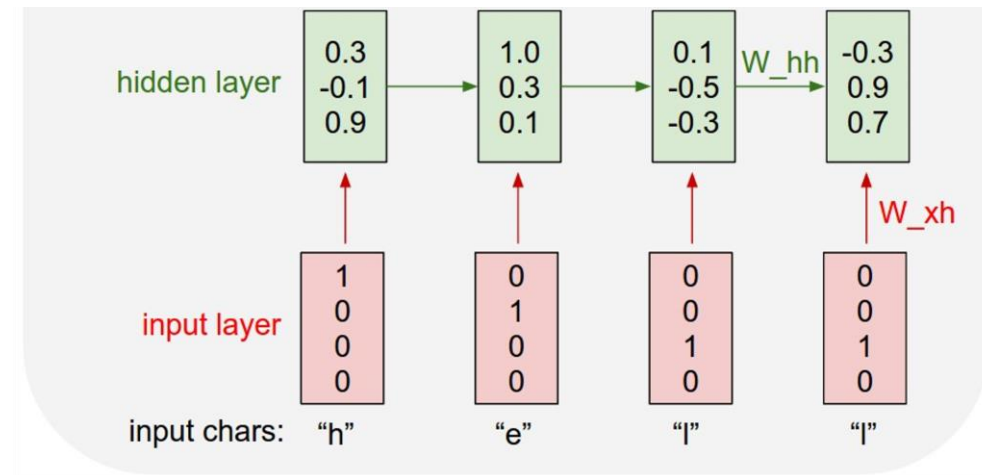
Training sequence: "hello"
Vocabulary: [h, e, l, o]



Example: language modeling

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

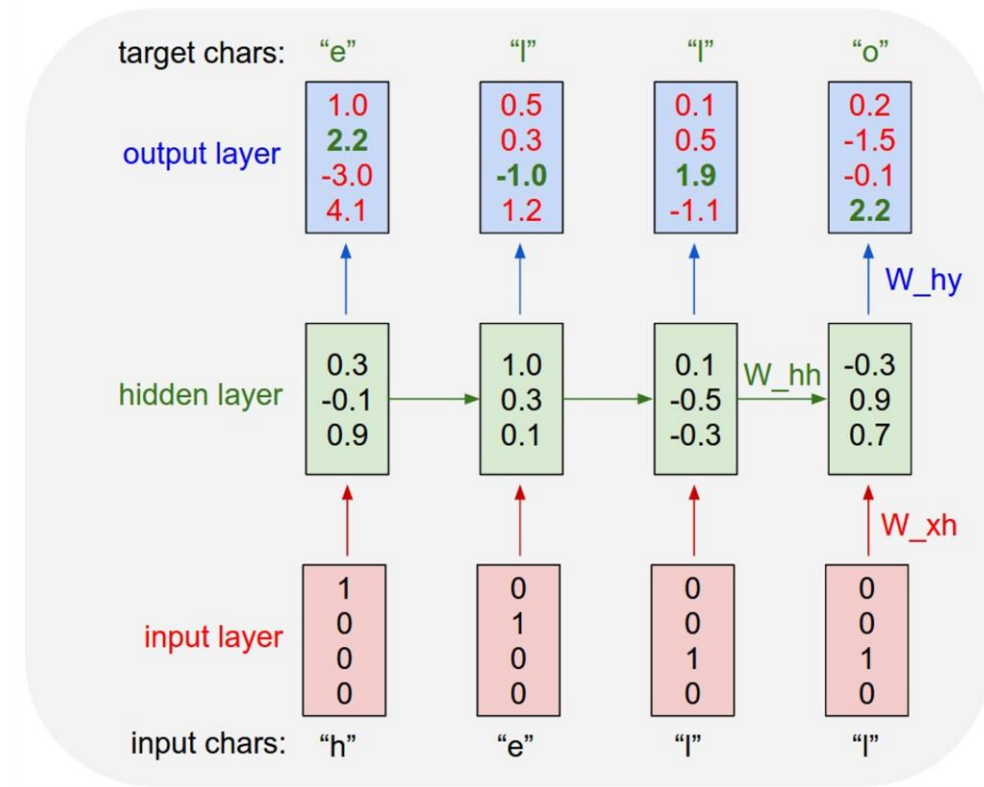


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

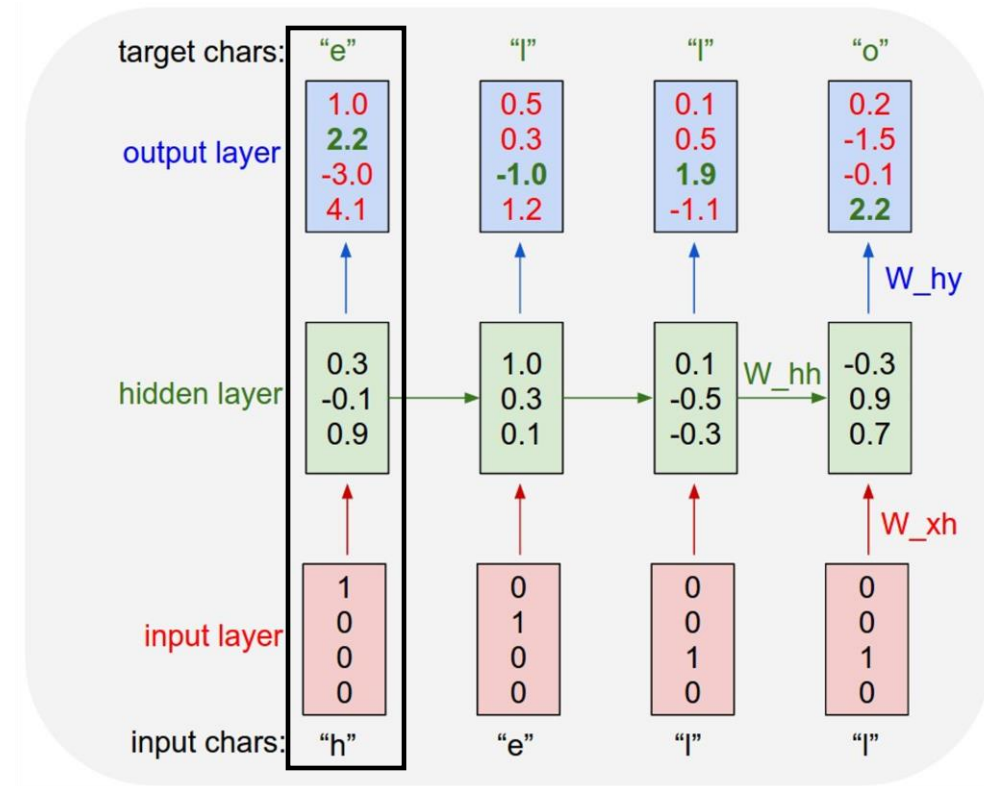


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

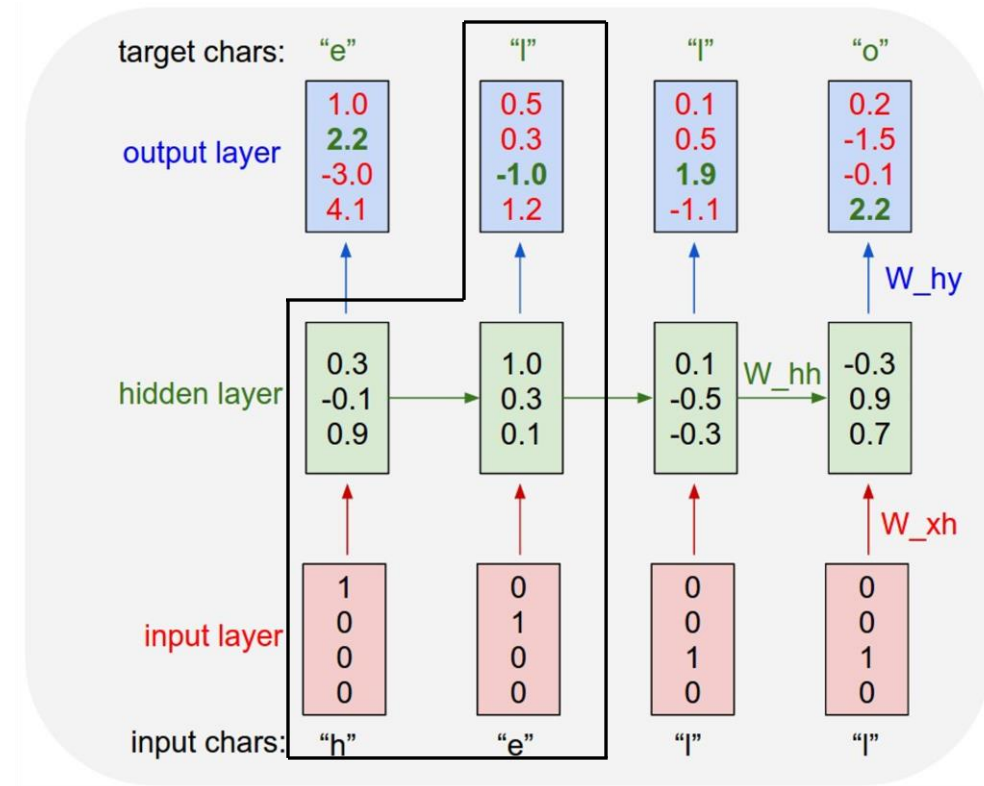


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

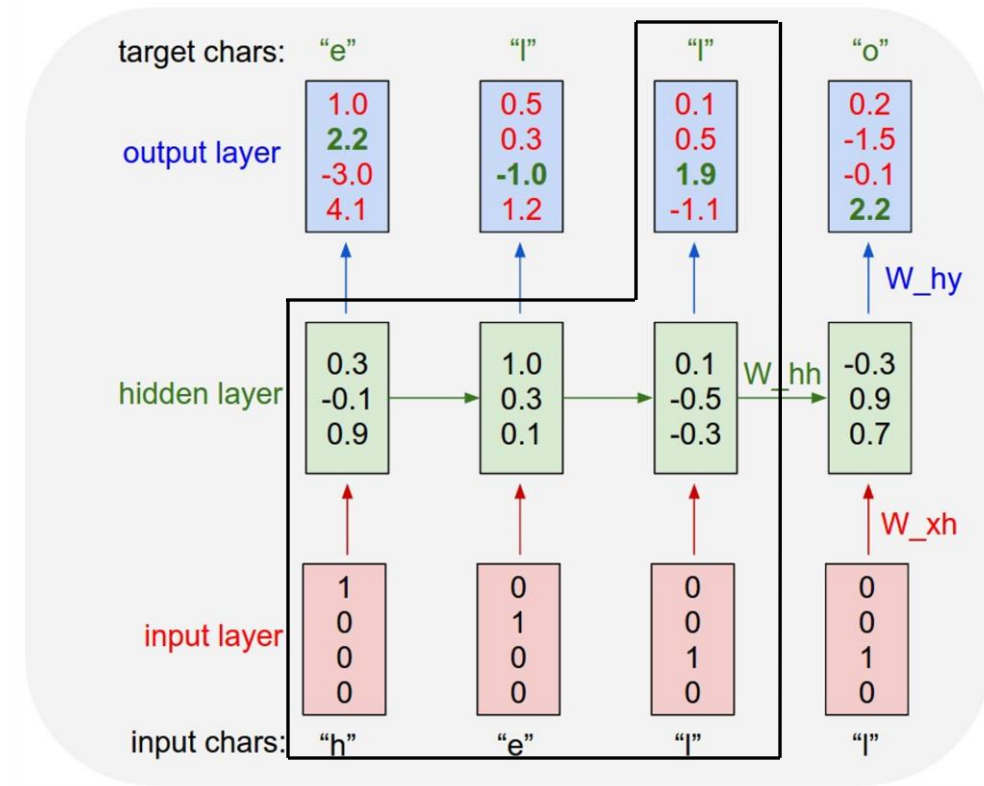


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

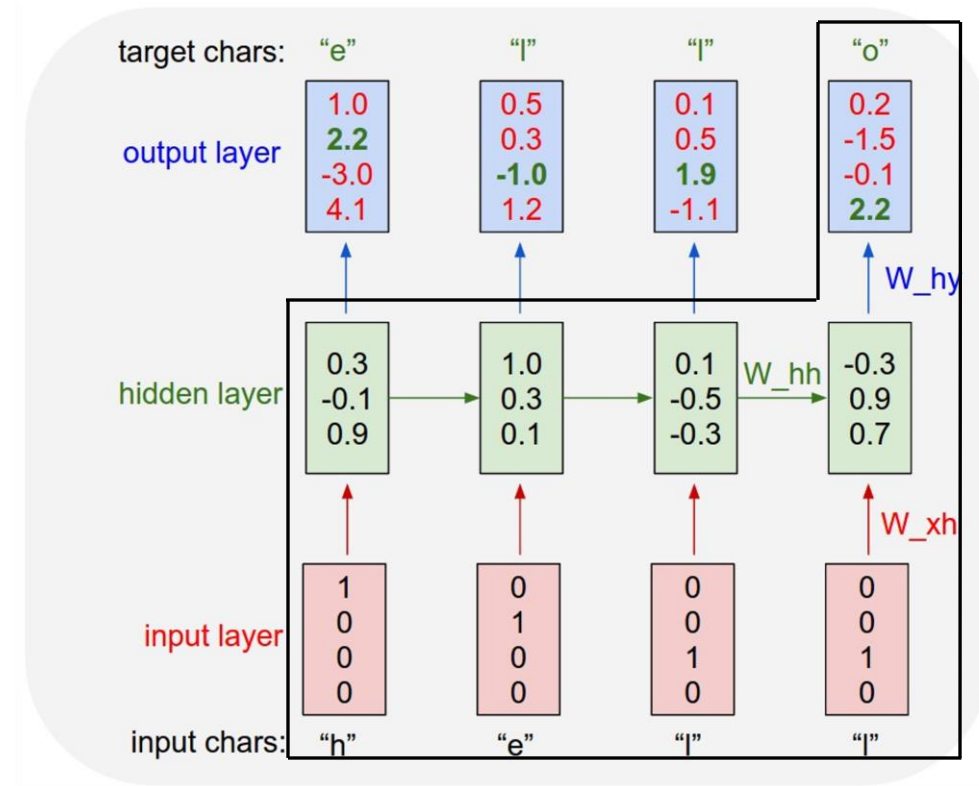


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

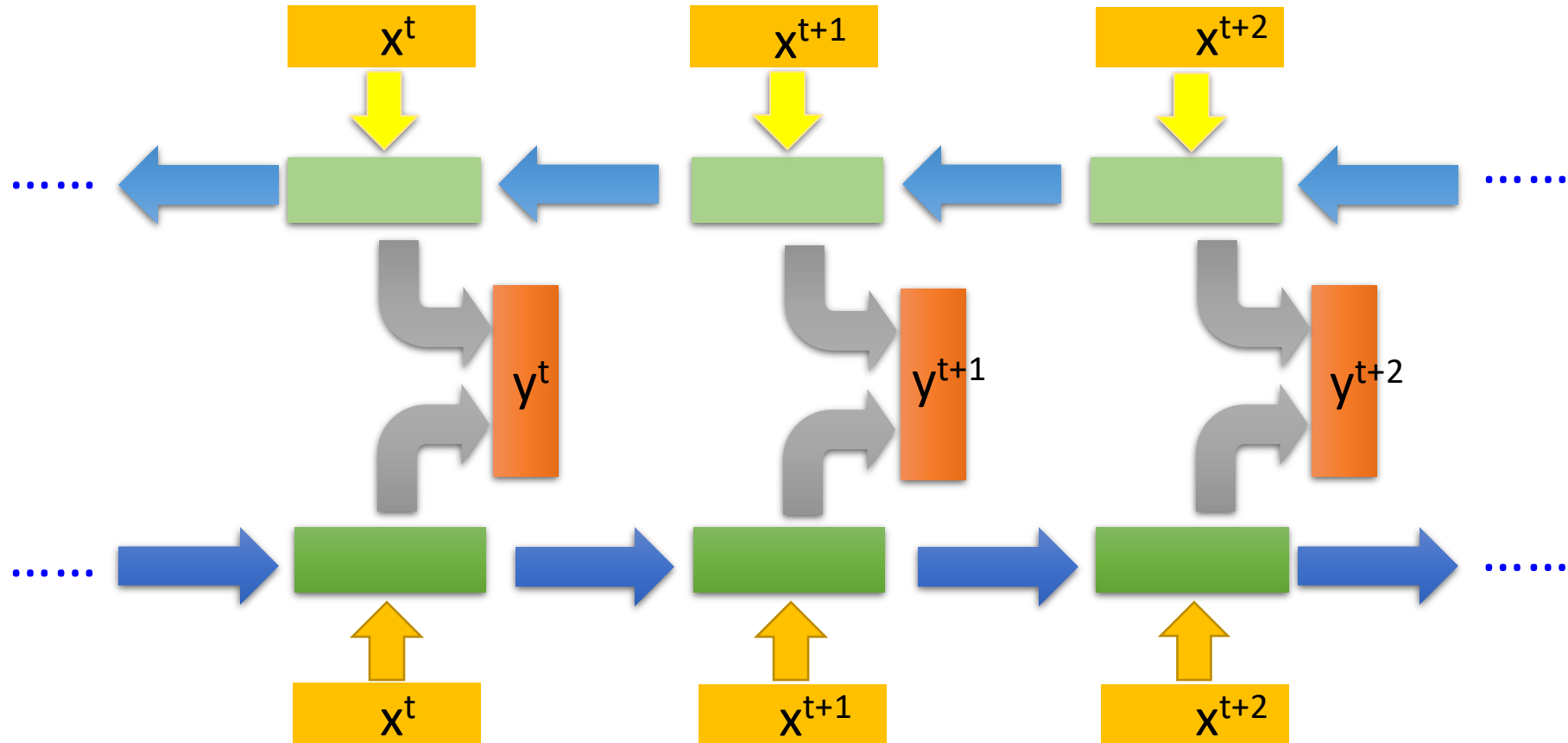
Training sequence: "hello"
Vocabulary: [h, e, l, o]



Bidirectional RNN

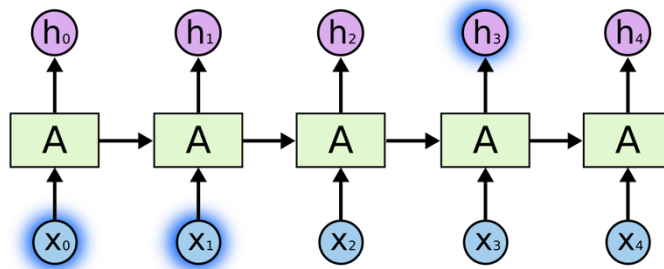
- Filling in the blank in a text sequence:
 - I am ____.
 - I am ____ hungry.
 - I am ____ hungry, and I can eat half a pig.
- The end of the phrase (if available) conveys significant information about which word to pick
- *Bidirectional RNNs*
 - Instead of running an RNN only in the forward mode starting from the first token, we **start another one from the last token** running from back to front.

Bidirectional RNN



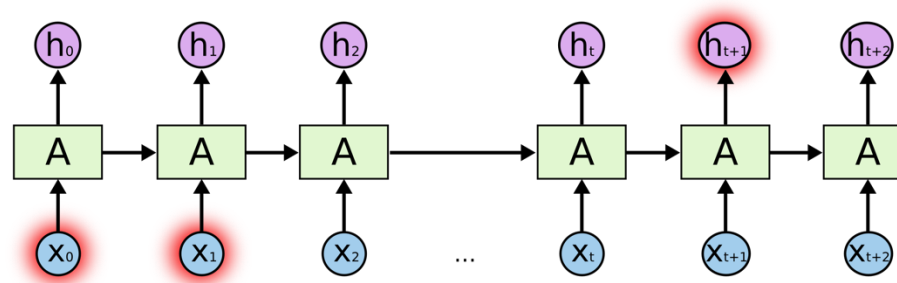
Problem of long-term dependencies

- RNNs are able to connect previous information to the present task



predict the last word in “the clouds are in the sky,”

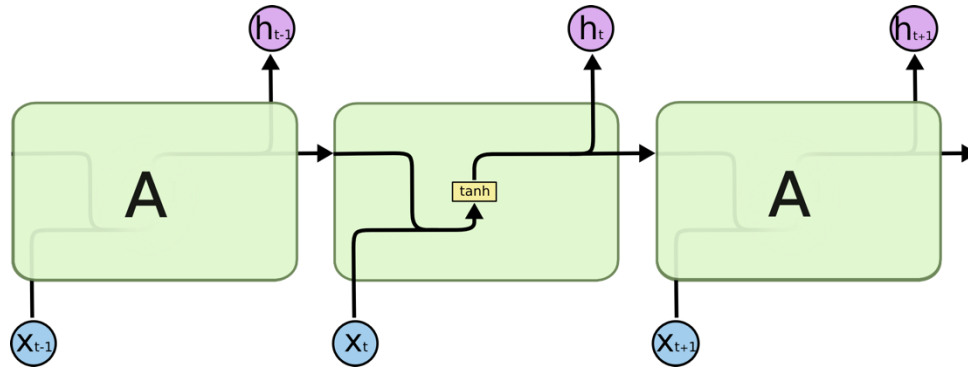
- Unfortunately, as that gap grows, RNNs become unable to learn to connect the information



predict the last word in the text “I grew up in France ... I speak fluent French.”

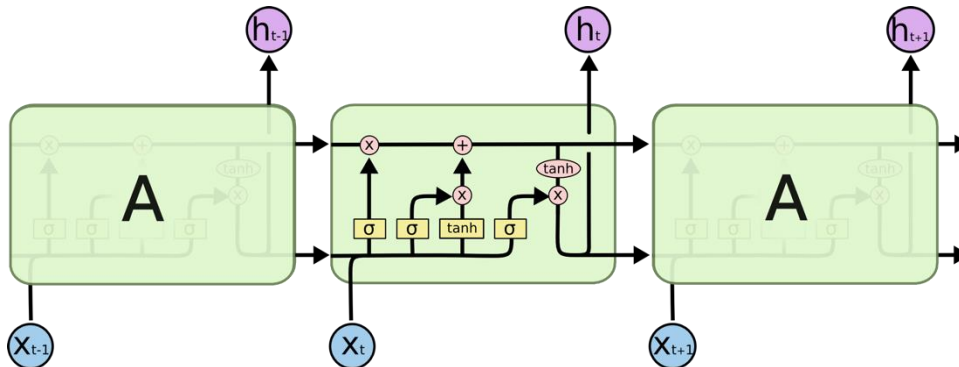
Long Short Term Memory (LSTM) networks

- Standard RNN



The repeating module in a standard RNN contains a single layer

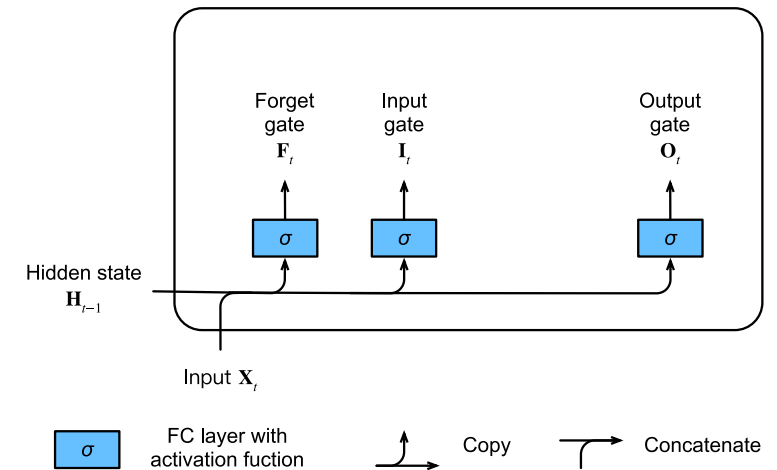
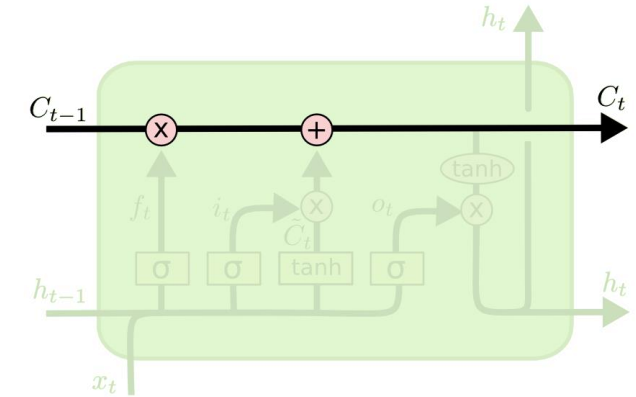
- LSTMs are explicitly designed to avoid the long-term dependency problem



The repeating module in an LSTM contains four interacting layers

Core idea behind LSTM

- Key to LSTMs is the *memory cell*
 - Has the same shape as the hidden state
 - Designed to record additional information
- To control the memory cell we need a number of gates; gates are a way to optionally let information through
 - *forget gate*: reset the content of the cell
 - *input gate*: decide when to read data into the cell. We refer to this as the
 - *output gate*: read out the entries from the cell

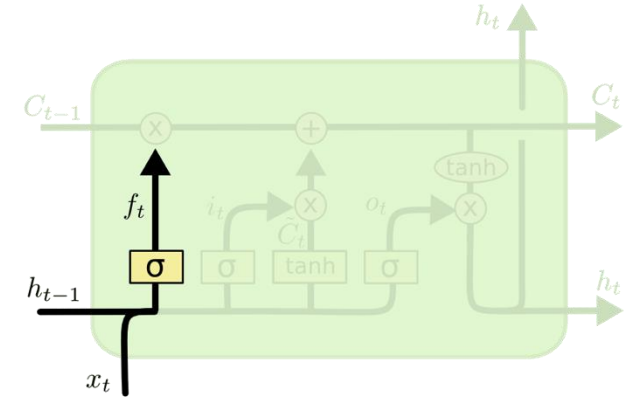


Step-by-step in LSTM

- What information to forget
- The first step in LSTM is to decide what information we're going to throw away from the cell state.
- The decision is made by a sigmoid layer called the **forget gate**
 - Input: previous state h_{t-1} and current input x_t
 - Output: a number between 0 and 1 applied on previous cell state C_{t-1}

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Decision
 - $f_t = 1$ represents “completely keep”
 - $f_t = 0$ represents “completely forget”



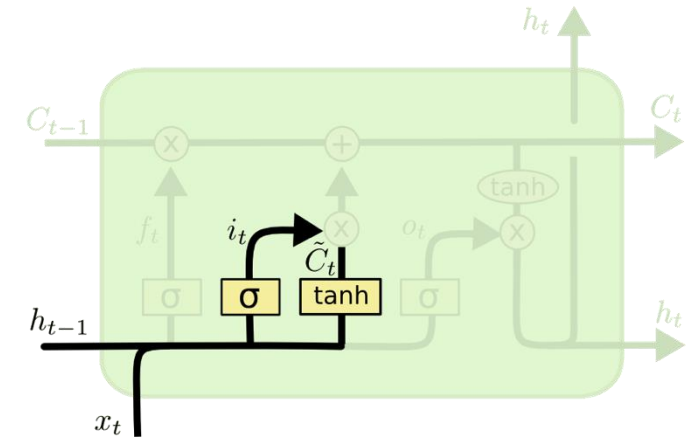
Step-by-step in LSTM

- What new information to store
- Decide what new information to store in the cell state
- First, sigmoid layer called **input gate** decides which values we'll update

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- Second, tanh layer creates a new **candidate memory cell**
 - It could be added to the state.

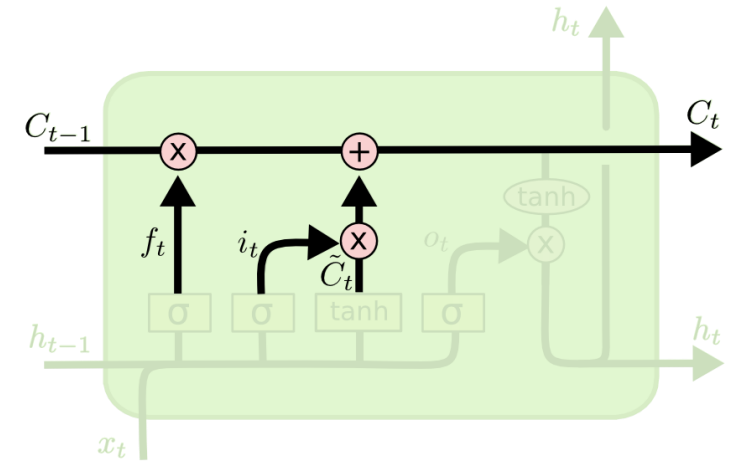
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



Step-by-step in LSTM

- **Update the cell state**
- Update old cell state C_{t-1} to new cell state C_t
- Part 1: Multiply the old state by f_t ,
 - forgetting the things we decided to forget earlier.
- Part 2: Add $i_t \times \tilde{C}_t$
 - $\tilde{C}_t$ is the new candidate values
 - i_t decides how much we decided to update each state value.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Step-by-step in LSTM

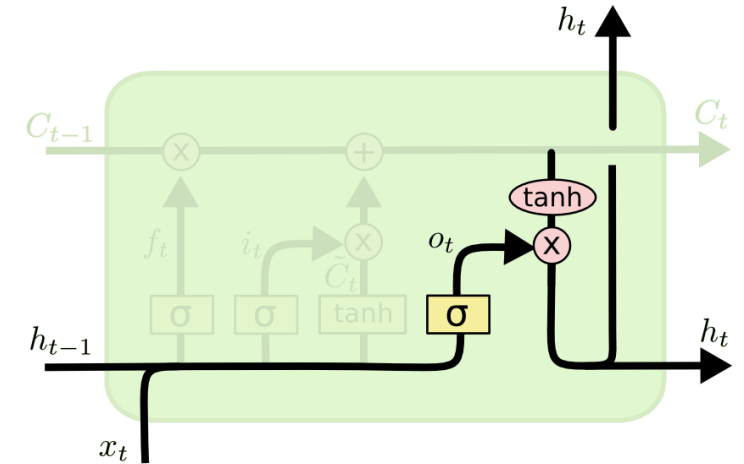
- What to output
- Goal: decide what to output for hidden state h_t
- Output o_t from the **output gate**

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

- Out put is the gated version of the tanh of the memory cell

$$h_t = o_t * \tanh(C_t)$$

- output gate close to 1: pass all memory information,
- output gate close to 0: we retain all the information only within the memory cell and perform no further processing



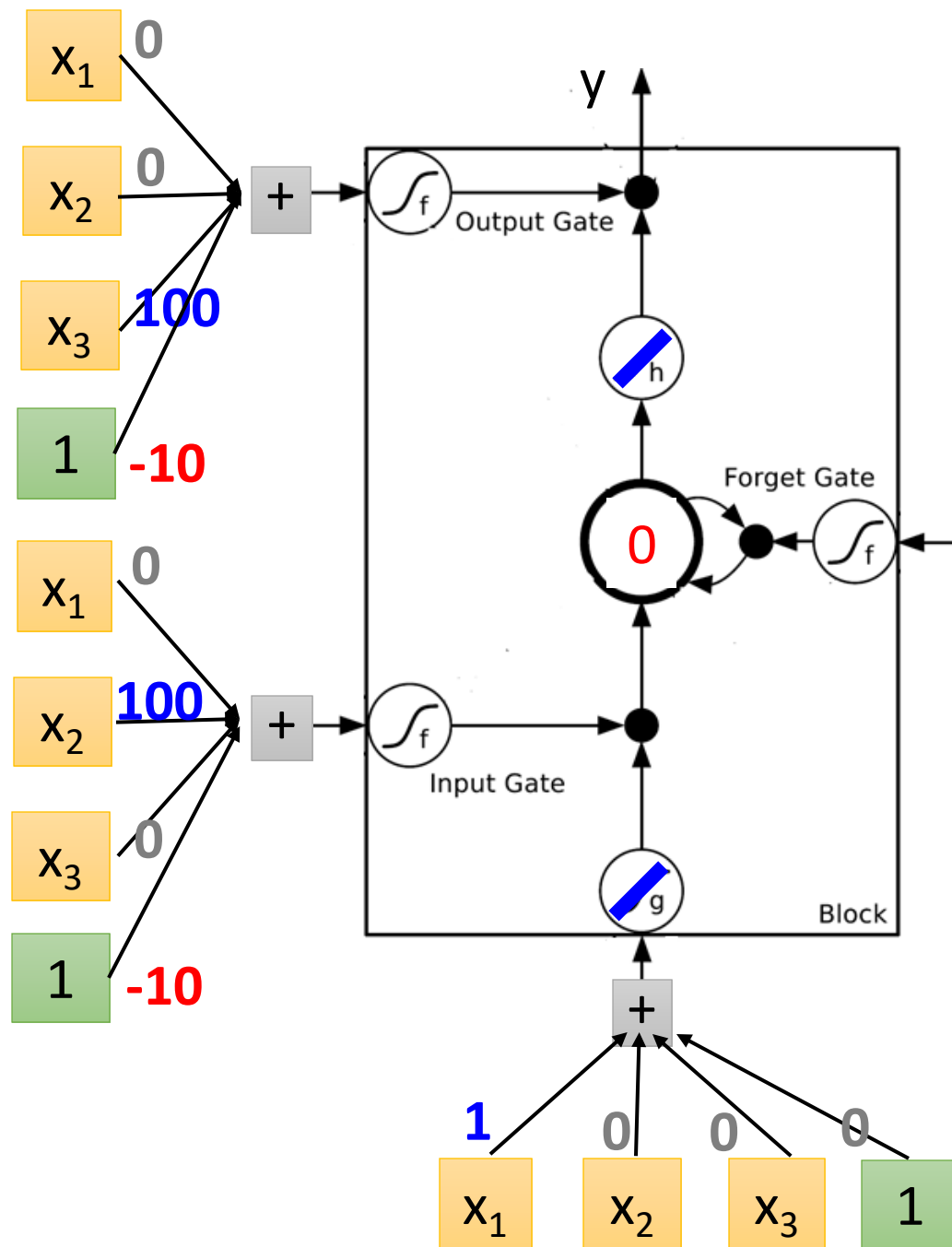
LSTM - Example

	0	0	3	3	7	7	7	0	6	Memory
x_1	1	3	2	4	2	1	3	6	1	
x_2	0	1	0	1	0	0	-1	1	0	
x_3	0	0	0	0	0	1	0	0	1	
y	0	0	0	0	0	7	0	0	6	

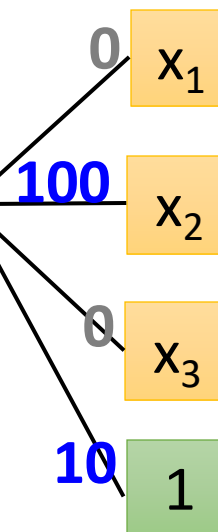
When $x_2 = 1$, add the numbers of x_1 into the memory

When $x_2 = -1$, reset the memory

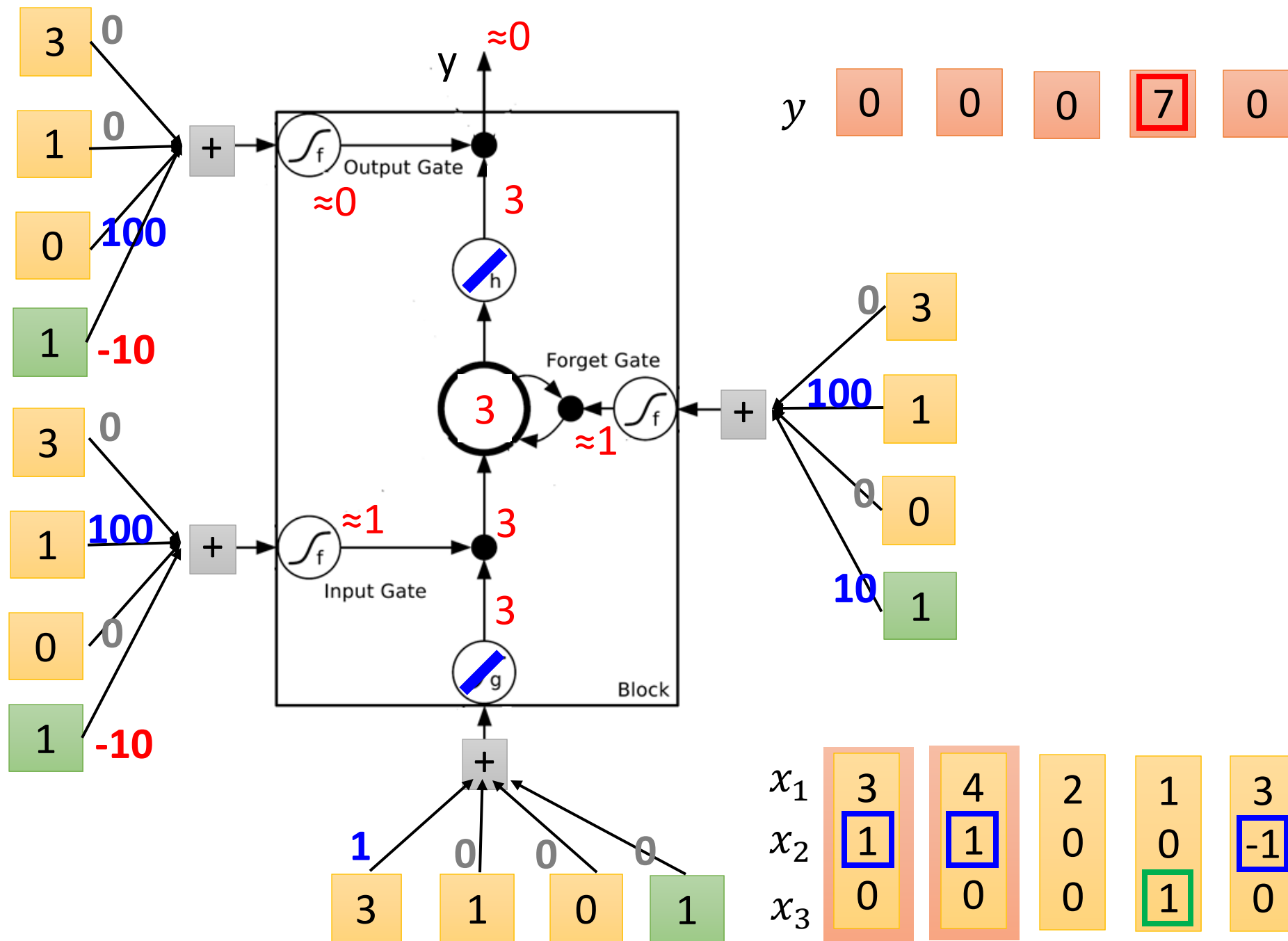
When $x_3 = 1$, output the number in the memory.

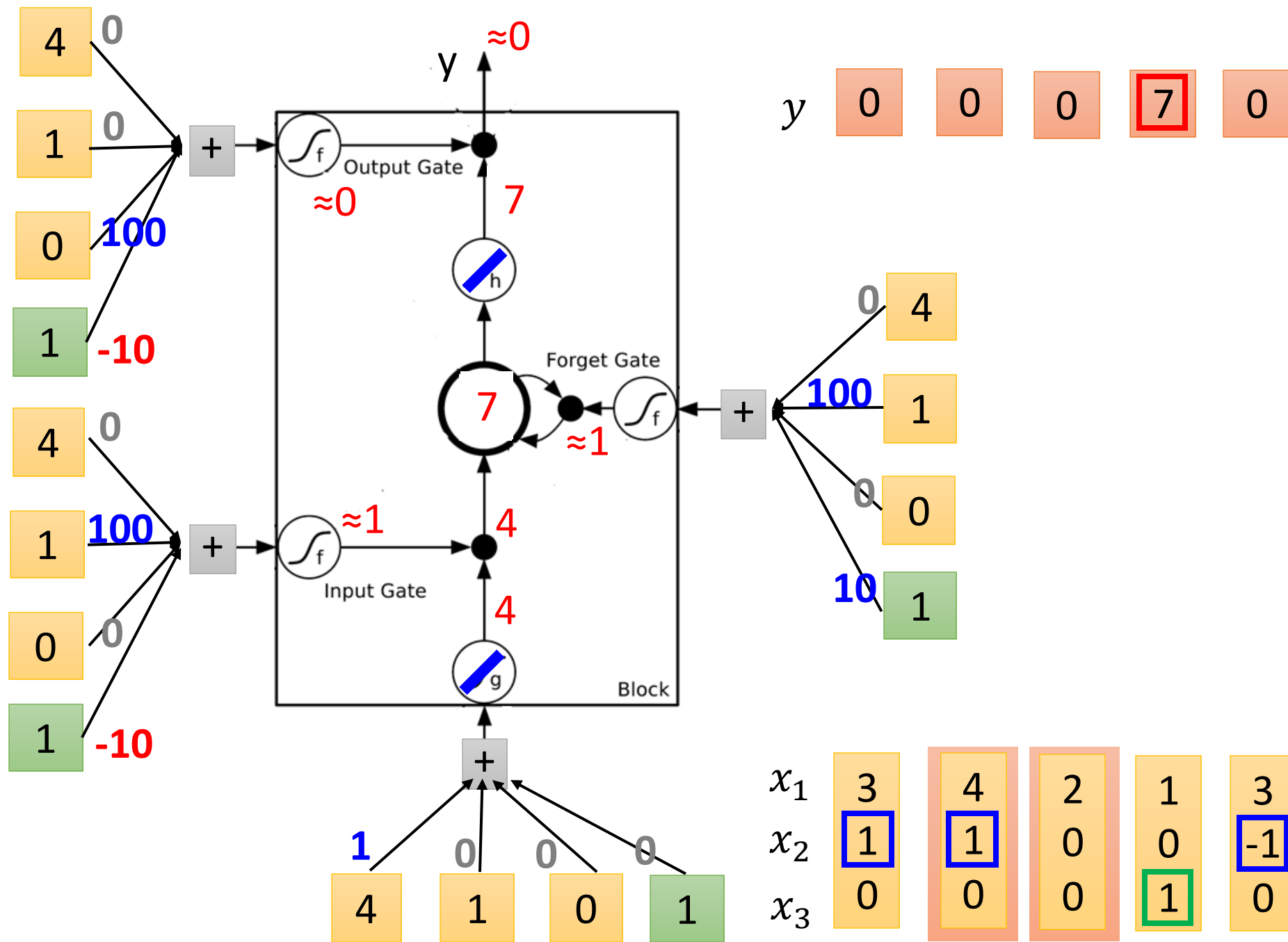


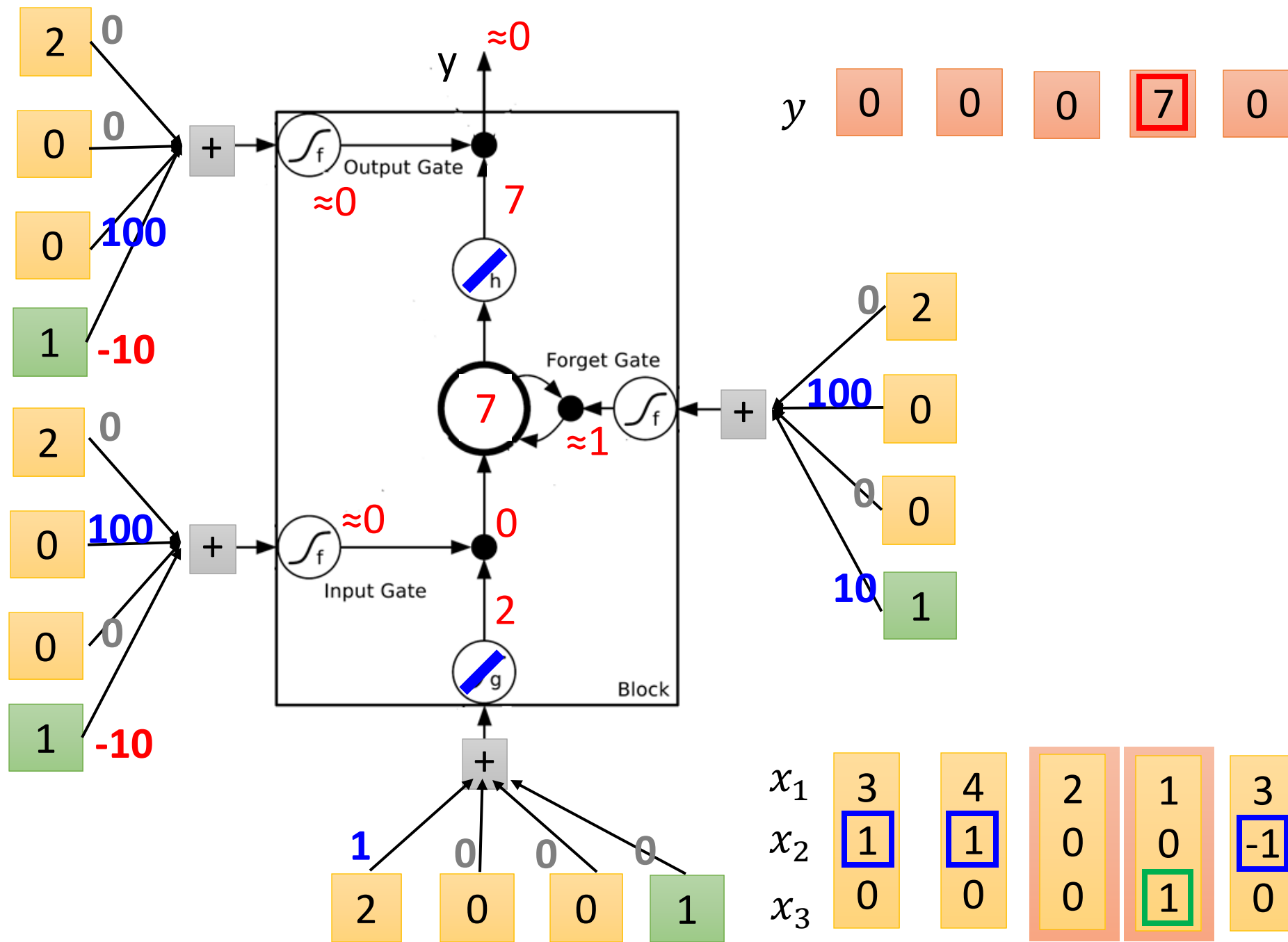
y 0 0 0 7 0

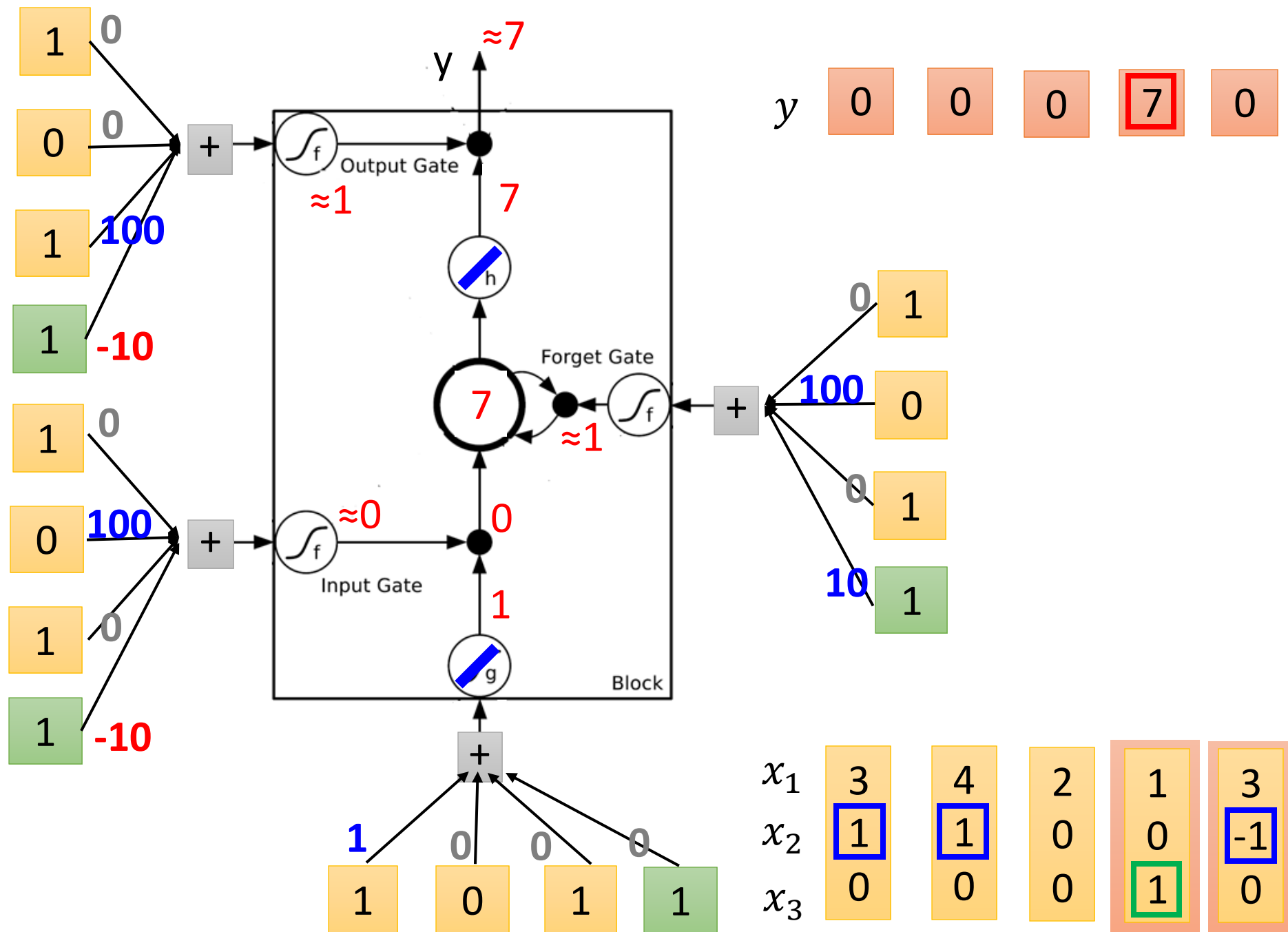


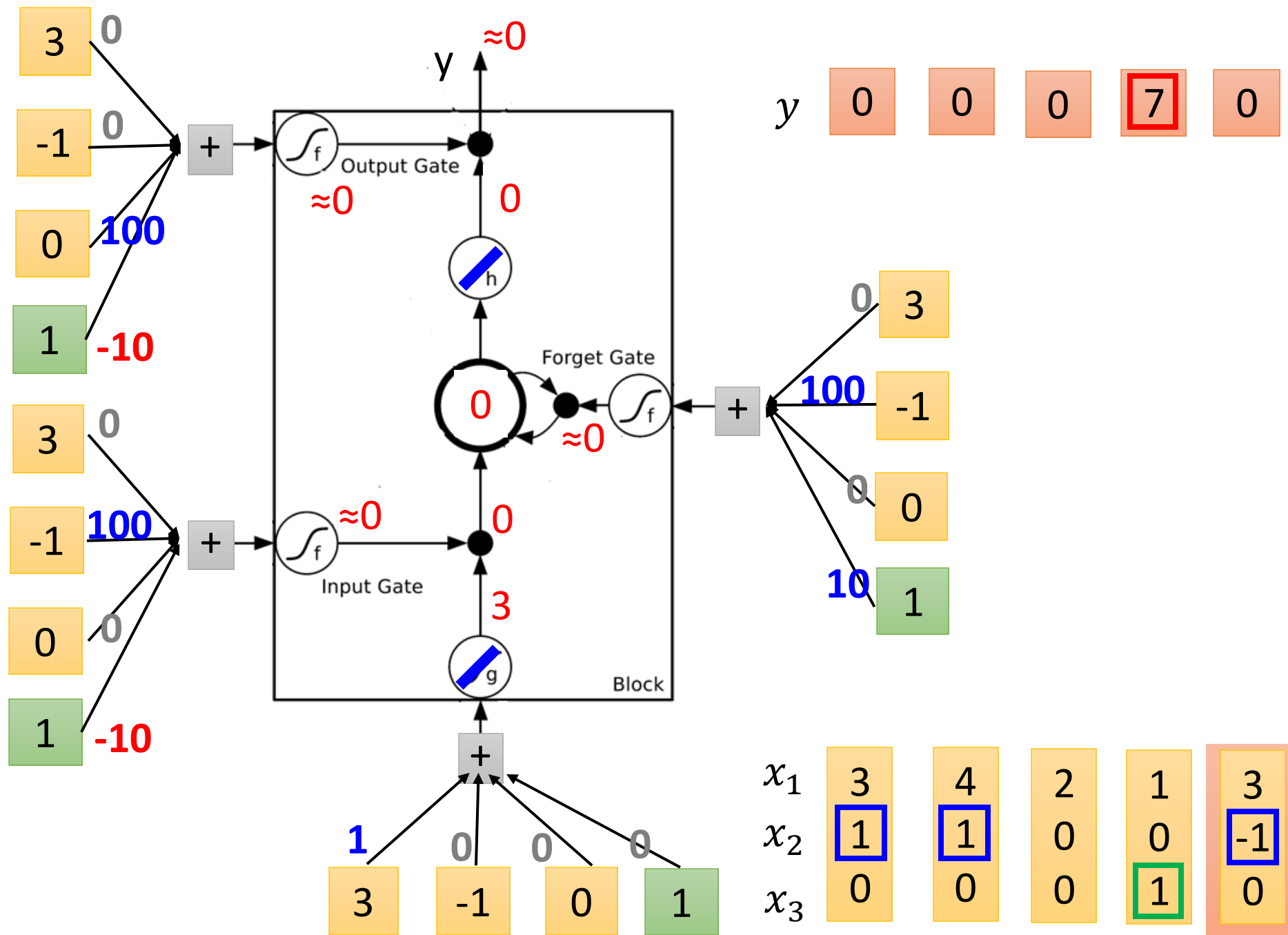
x_1 3 4 2 1 3
 x_2 1 1 0 0 -1
 x_3 0 0 0 1 0



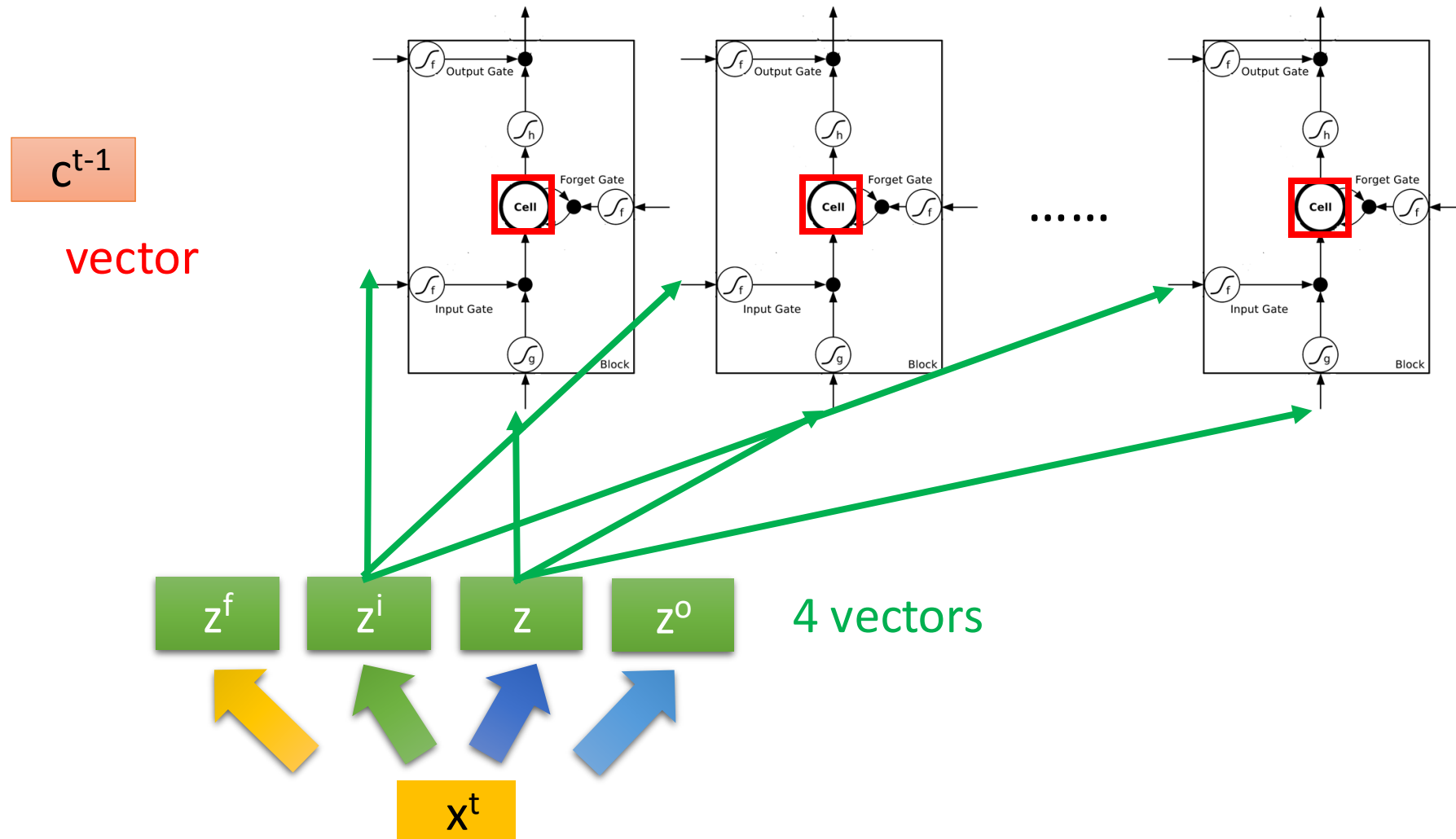




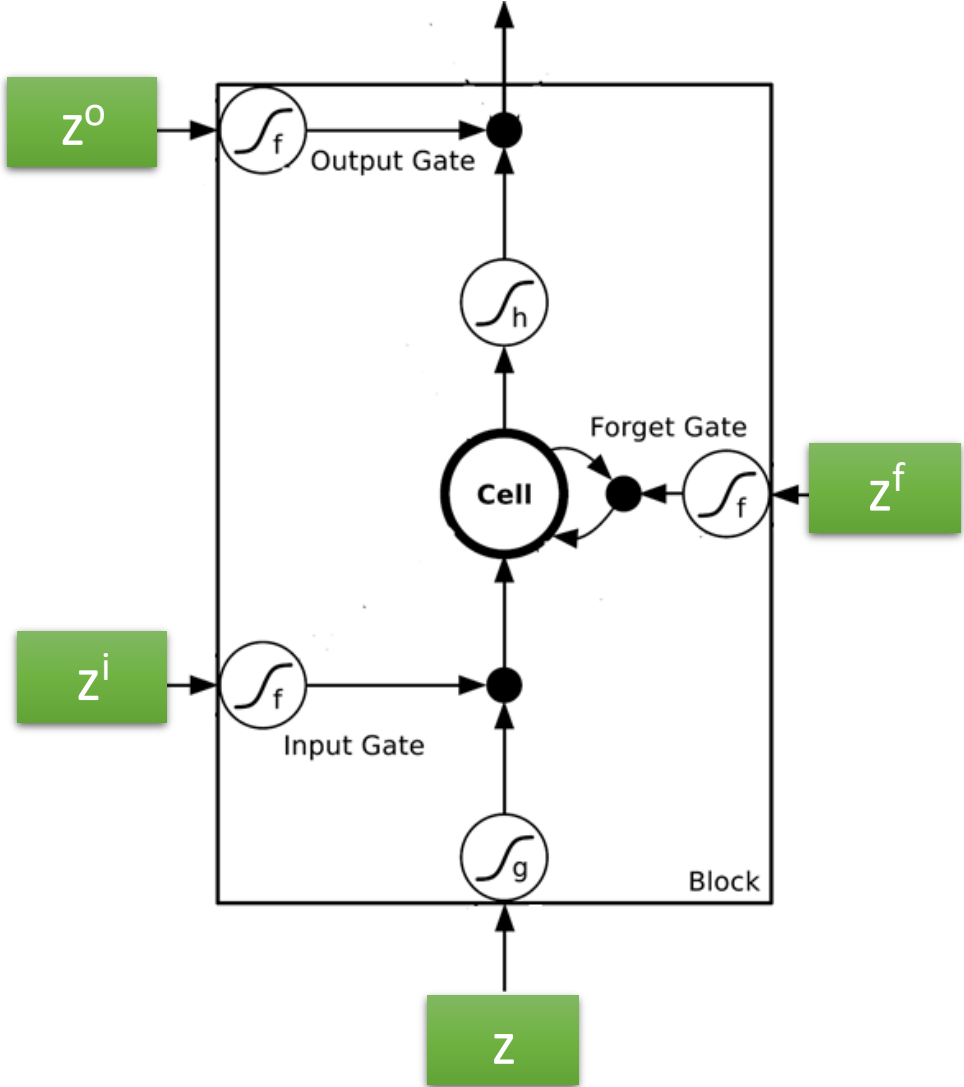
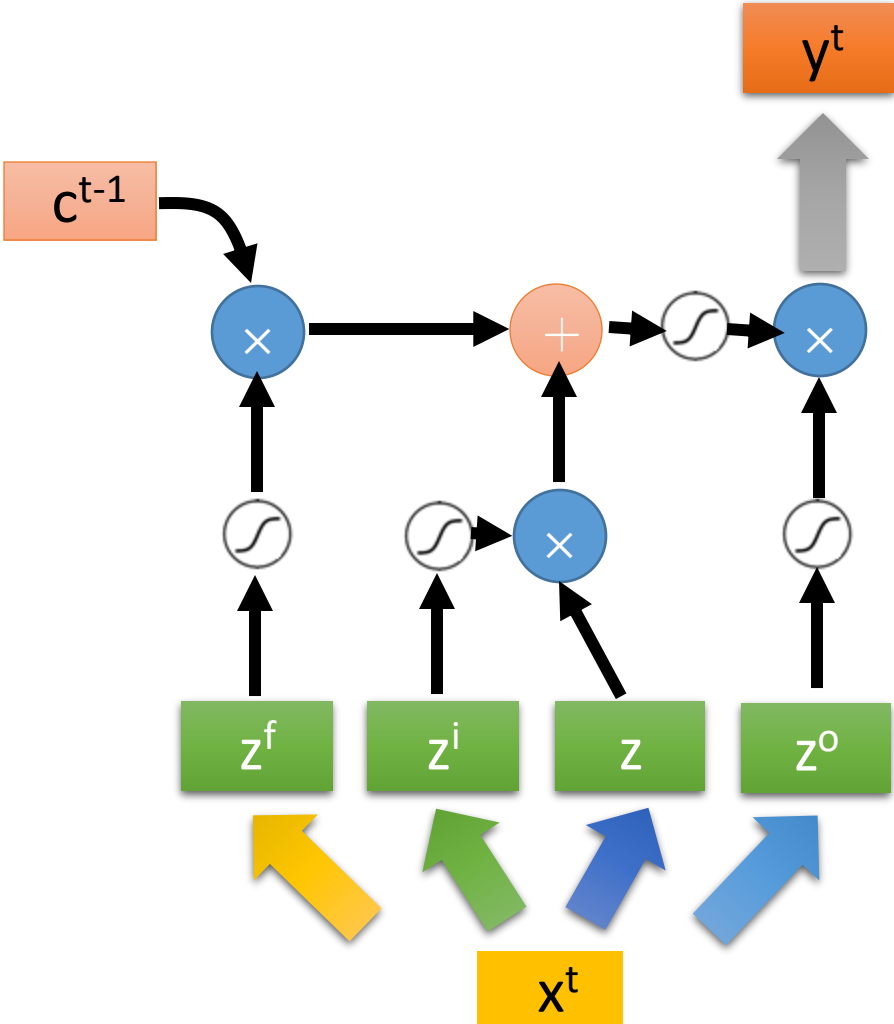




LSTM



LSTM



Gated Recurrent Unit (GRU)

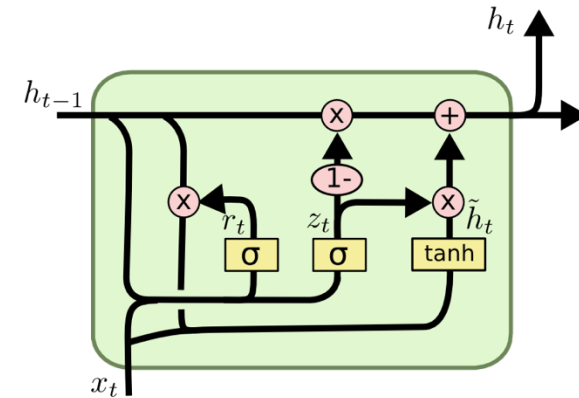
- GRU combines the forget and input gates into a single “*update gate*”
- GRU merges the cell state and hidden state
- GRU is *simpler* than standard LSTM models, and has been growing increasingly popular

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

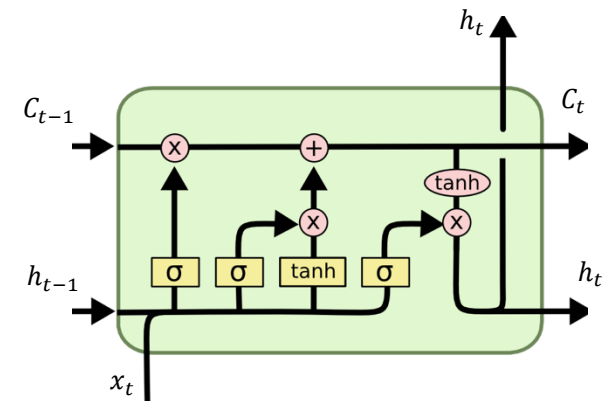
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



GRU



LSTM

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don't work very well
- Common to use LSTM and GRU: additive interactions improve gradient flow
- RNNs can go deep
- Wide applications in NLP